Supervised Machine Learning
Regression and Classification

These notes are compiled for personal learning purposes based on the course content.

Contents

Module 1: Introduction to Machine Learning

1 Overview of Machine Learning

What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that focuses on enabling computers to learn from data and improve their performance over time without being explicitly programmed. Unlike traditional software, which follows fixed instructions, ML systems identify patterns and make decisions based on examples and experience.

Many real-world problems are too complex for rule-based programming. Machine learning offers flexible and scalable solutions for such tasks by training algorithms to automatically extract useful information from data.

Machine learning technologies are embedded in many daily digital interactions, often operating in the background without users being aware. Common examples include:

(1) **Search Engines:** When a user searches for information on platforms like Google, ML algorithms help rank and suggest the most relevant pages based on previous user behavior.

(2) **Social Media:** Applications such as Instagram and Snapchat use ML to recognize faces, recommend content, and filter harmful or unwanted messages.

(3) **Streaming Platforms:** Services like Netflix and YouTube generate personalized content recommendations by analyzing viewing patterns and preferences.

(4) **Voice Assistants:** Digital assistants like Siri, Alexa, or Google Assistant interpret spoken language using ML techniques to perform tasks or answer questions.

(5) **Email Spam Filters:** Email systems detect and divert spam messages using models trained on large datasets of legitimate and unwanted emails.

Machine Learning in Industry and Science

Beyond personal use, machine learning has become essential across various industries and research fields:

(1) **Healthcare:** ML helps in diagnosing diseases, analyzing medical images, and recommending personalized treatments.

(2) **Climate and Energy:** Renewable energy systems, like wind farms, use ML to forecast energy production and optimize performance.

(3) **Finance:** Banks and financial institutions use ML to detect fraudulent transactions, assess credit risk, and automate trading.

(4) **Manufacturing:** Predictive maintenance systems powered by ML help identify faults before they occur, reducing downtime and costs.

Looking Ahead: Artificial General Intelligence

One of the ultimate goals of AI research is the development of **Artificial General Intelligence (AGI)**, a system with the ability to understand, learn, and apply knowledge across a wide range of tasks at a human level.

- AGI is still a long-term objective. Many experts believe it may take several decades to become a reality.
- The path to AGI largely depends on improving current learning algorithms so machines can adapt to new situations and generalize knowledge beyond specific tasks.

2 Supervised vs. Unsupervised Machine Learning

2.1 What Is Machine Learning?

A well-known early definition of machine learning was introduced by Arthur Samuel in 1959:

> *"The field of study that gives computers the ability to learn without being explicitly programmed."*

Arthur Samuel developed one of the earliest machine learning programs: a checkers-playing computer system.

↪ Samuel was not a checkers expert. Instead of hard-coding strategies, he enabled the program to learn from experience.

↪ The system played thousands of games against itself, identifying which board positions led to wins or losses.

↪ Over time, it improved by favoring successful patterns and avoiding ineffective ones.

Three Main Types of Machine Learning

- **Supervised Learning**
  - Most commonly used in real-world applications
  - Requires labeled data (input-output pairs)
  - Enables models to learn to map inputs to correct outputs
  - Used in tasks like image classification, speech recognition, and forecasting

- **Unsupervised Learning**
  - Used when data lacks labels
  - Helps uncover hidden patterns or groupings in data
  - Commonly applied to clustering and anomaly detection
  - Supports tasks like customer segmentation and dimensionality reduction

- **Reinforcement Learning**
  - Involves an agent interacting with an environment
  - Learns through trial and error using rewards and penalties
  - Aims to maximize long-term cumulative reward
  - Applied in robotics, game playing, and autonomous systems

Figure 1 shows an overview of the main categories of machine learning.

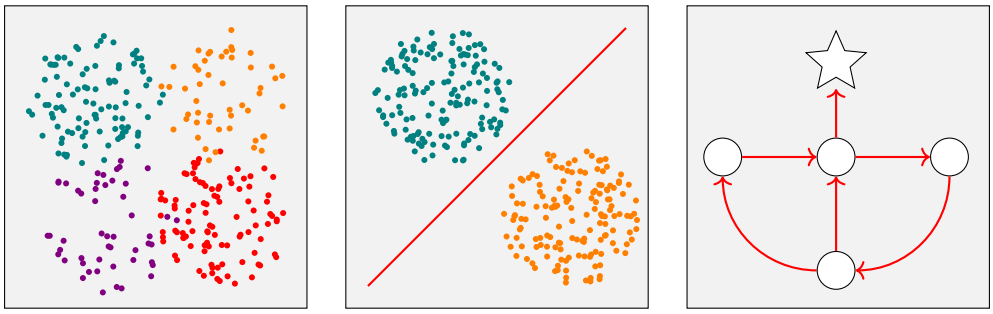


FIGURE 1. Overview of the main categories of machine learning.

2.2 Supervised Learning Part 1

What Is Supervised Learning?

Supervised learning trains an algorithm to map **input (X)** to **output (Y)** using labeled examples. By learning from many input-output pairs, the algorithm can make accurate predictions on new, unseen data, as illustrated in Figure 2.

Examples of Supervised Learning Applications

(1) **Spam Detection**
- *Input:* An email
- *Output:* Spam or Not Spam

(2) **Speech Recognition**
- *Input:* Audio clip
- *Output:* Text transcript

(3) **Machine Translation**
- *Input:* English sentence
- *Output:* Translated sentence in Spanish, Arabic, Hindi, etc.

(4) **Online Advertising**
- *Input:* Information about an ad and the user
- *Output:* Probability the user will click the ad

(5) **Self-Driving Cars:**
- *Input:* Images from cameras and sensor data
- *Output:* Position of nearby cars and objects

(6) **Visual Inspection in Manufacturing:**
- *Input:* Photo of a product (e.g., smartphone)
- *Output:* Presence or absence of a defect

In all these cases, the model is trained using correct labels. Once trained, it can generalize to make predictions on new data.

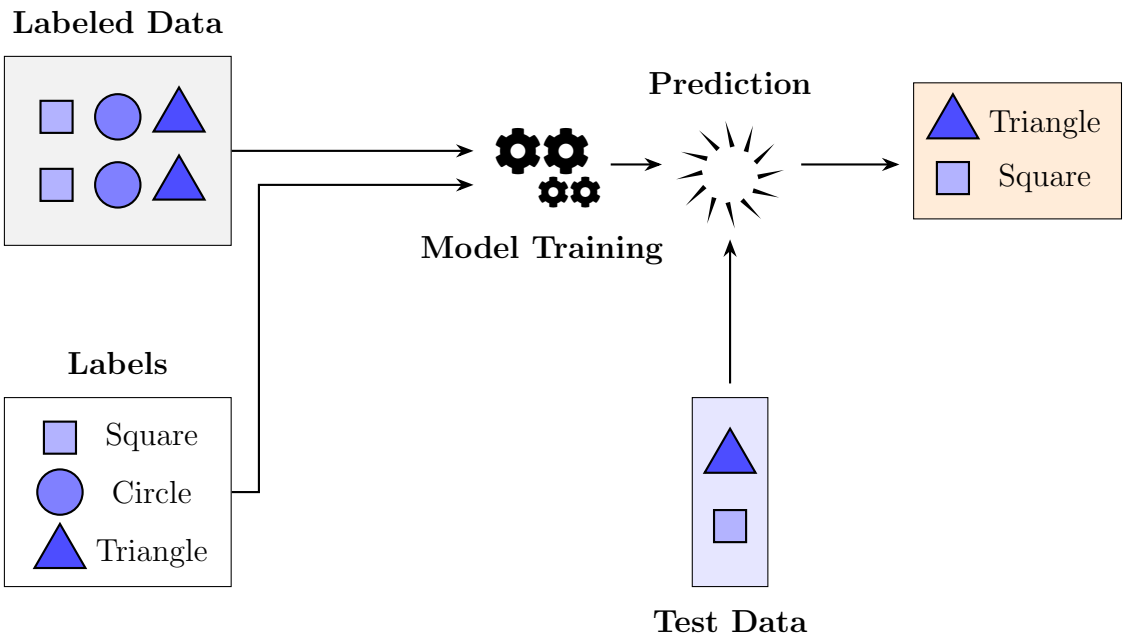


FIGURE 2. Supervised learning: training on labeled data to predict unseen inputs.

Example: Predicting House Prices

One common supervised learning task is estimating house prices based on property features such as size.

- Suppose a model is trained on examples of house sizes and their sale prices.

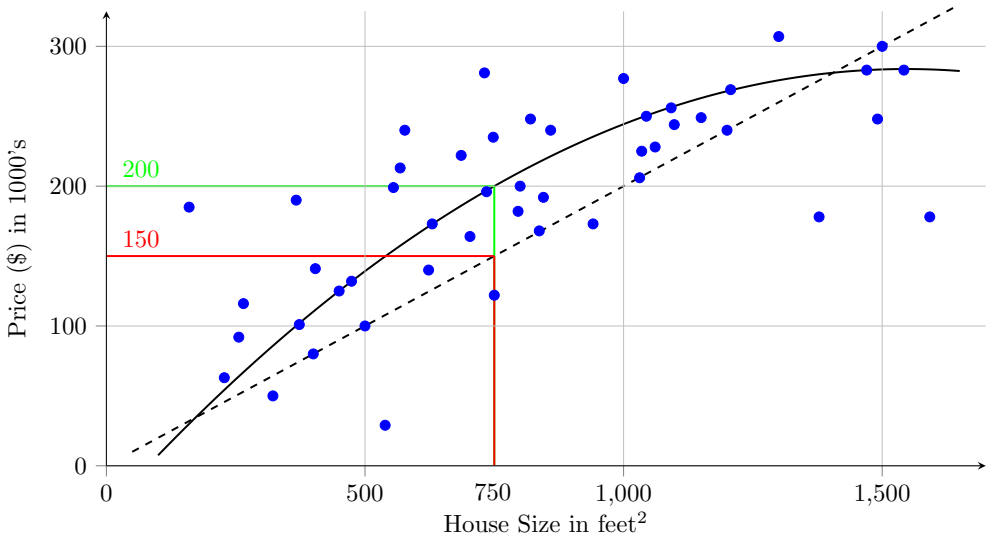


FIGURE 3. House price vs. size with linear and polynomial regression models.

- When asked to predict the price of a 750 ft^2 house:
  - A simple algorithm may fit a straight line through the data and predict a price of around $150K.
  - A more complex model might fit a curve, predicting a price closer to $200K.
- Different models make different predictions. The task of the learning algorithm is to choose the most appropriate function (line or curve) to fit the data.

This example is called a **regression problem** (a type of supervised learning where the goal is to predict a numerical value).

2.3 Supervised Learning Part 2

Supervised learning algorithms learn how to predict an output based on a given input. In other words, they learn the relationship between X (input) and Y (output).

- One type of supervised learning is called a **regression algorithm**. It is used to predict a *number* from an infinite range of possible values. For example, predicting the price of a house based on its size and location.

- Another important type of supervised learning is called a **classification algorithm**. This type of algorithm is used when the output is a *category or label* instead of a number. For example, the prediction could be `yes` or `no`, `cat` or `dog`, or one of several possible classes

Example: Breast Cancer Diagnosis

Suppose you are building a machine learning system to assist doctors in detecting breast cancer. Early detection is important because it can potentially save lives. Using a patient's medical records, the system attempts to determine whether a tumor (a lump) is:

- **Malignant:** cancerous and dangerous
- **Benign:** non-cancerous and not dangerous

Consider a dataset in which each tumor is described by its size. The horizontal axis represents the tumor size, while the vertical axis indicates the diagnosis:

`0 = benign  1= malignant`

This is a **classification problem** because the output is limited to two distinct categories.

↪ In Figure 4, each data point represents a tumor.

↪ Each point shows both the size of the tumor and its diagnosis, which is either `benign (0)` or `malignant (1)`.

↪ Classification problems like this are different from regression because they involve a small set of possible outputs (*not continuous values*).

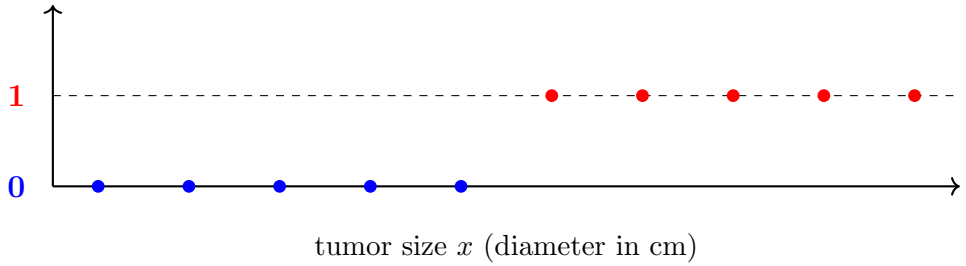


FIGURE 4. Tumor size vs. diagnosis (0 = benign, 1 = malignant).

Multi-class Classification

- Multi-class classification refers to supervised learning tasks in which each input is categorized into one of three or more distinct classes. Unlike binary classification, which distinguishes between only two classes.

- In real world classification tasks, using only one input feature is often not enough to make accurate predictions. Models typically consider multiple features such as tumor size, patient age, or cell characteristics to better differentiate between classes.
  - When multiple features are included, the model works in a higher dimensional space. This allows it to learn more precise decision boundaries, which improves classification performance, especially in complex situations where the classes may overlap.

Figure 5 illustrates how using two features can help separate benign and malignant tumors more effectively. A new patient's data point (e.g., represented by a green marker) can be placed in this space, and the model predicts the class based on the learned boundary.

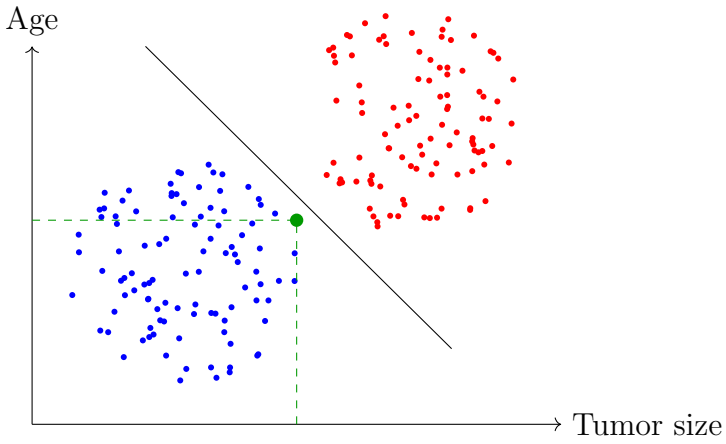


FIGURE 5. Tumor classification based on age and tumor size with a nonlinear decision boundary.

2.4 Unsupervised Learning Part 1

Unsupervised learning deals with datasets that do not include labeled outputs. The purpose is not to predict specific outcomes but to uncover hidden patterns or structures within the data.

- The algorithm works with input data only; no target values are provided.
- It examines the features to find meaningful patterns or natural groupings.
- One widely used method is **clustering**, where the algorithm groups similar data points based on shared characteristics.

Figure 6 illustrates this process, where unlabeled inputs are organized into meaningful groups by an unsupervised algorithm.

Examples of Clustering Applications

Clustering, a key technique in unsupervised learning, is widely applied across diverse fields. The following examples highlight how it uncovers structure in unlabeled data

- **News Article Grouping (e.g., Google News):**
  Clustering algorithms automatically group related news articles by analyzing headlines and shared keywords. For example, articles mentioning terms like *panda*, *zoo*, and *birth* may be grouped under a topic such as "Panda gives birth," without human input. This approach helps organize thousands of daily news stories efficiently.

- **Genetic Data Analysis:**
  In bioinformatics, clustering is often used to group individuals based on DNA microarray data. A clustering algorithm identifies individuals with similar patterns of gene activity and groups them together, without relying on predefined categories or labels. A clustering algorithm identifies individuals with similar patterns of gene activity and groups them together, without relying on predefined categories or labels.

- **Customer Segmentation:**
  Clustering is often used in business to group customers into meaningful segments based on their behavior and demographic information. These customer segments can help support personalized marketing and targeted content delivery. This is done without using any labeled training data.

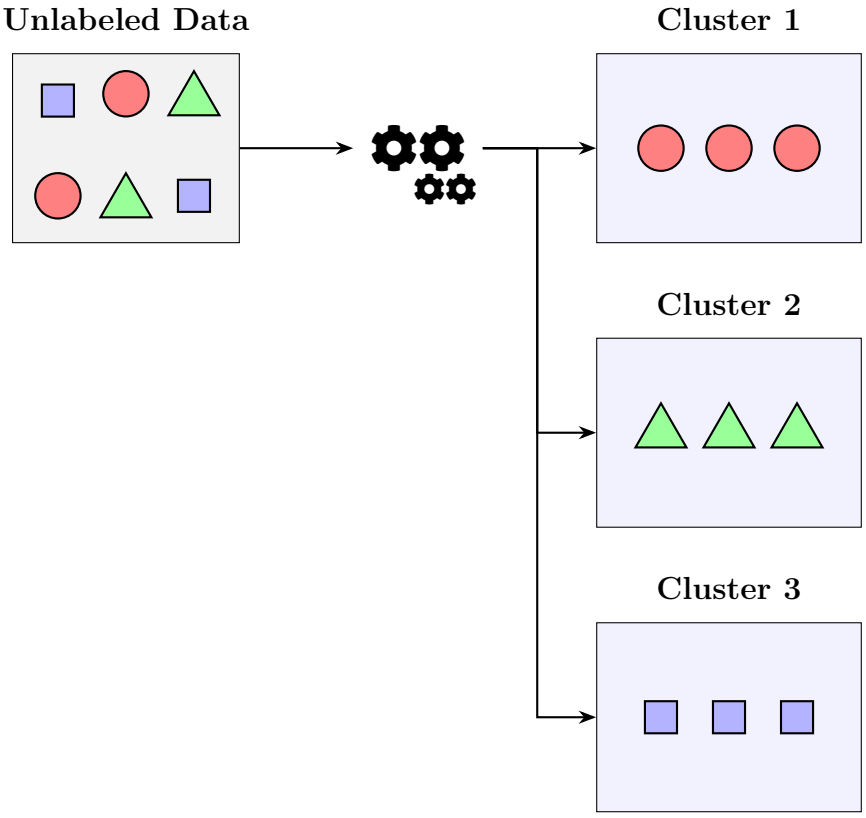


FIGURE 6. Unsupervised learning: clustering unlabeled data into distinct groups based on structure in the data.

2.5 Unsupervised Learning Part 2

Unsupervised learning refers to machine learning tasks where the dataset consists only of input features (x), without any corresponding output labels (y). The primary objective is to uncover hidden structures or patterns within the data.

While clustering is a well-known example, unsupervised learning encompasses a variety of additional techniques:

1. Anomaly Detection Anomaly detection focuses on identifying data points that deviate significantly from the norm. These unusual patterns often signify important or critical events. Applications include:

- **Fraud detection:** Identifying irregular financial transactions that may indicate fraudulent activity.
- **Cybersecurity:** Detecting network anomalies that could signal security breaches or attacks.
- **Industrial monitoring:** Finding deviations in machine behavior that may suggest faults or the need for maintenance.

2. Dimensionality Reduction Dimensionality reduction techniques are used to reduce the number of features in a dataset while retaining as much relevant information as possible. These methods are useful for:

- **Data visualization:** Projecting complex, high-dimensional data into two or three dimensions for easier interpretation.
- **Noise reduction:** Eliminating irrelevant or redundant features to improve model performance.
- **Computational efficiency:** Simplifying large datasets to reduce memory and processing requirements in machine learning pipelines.

2.6 Jupyter Notebooks

One of the most widely used tools among data science practitioners is the **Jupyter Notebook**. It is the default environment for developing and experimenting with machine learning models and is heavily adopted across industry, research, and education.

What Is a Jupyter Notebook?

A Jupyter Notebook is an interactive environment that combines **text**, **code**, and **visual output** in a single document. It supports multiple programming languages, with Python being the most widely used in data science applications. Key features include:

- **Markdown cells** allow users to write formatted text for explanations, documentation, and instructions.
- **Code cells** contain executable code. Python code can be run using `Shift + Enter`, and the output appears directly below the cell.

- The notebook supports LaTeX-style math, data visualizations, charts, interactive widgets, and inline plots.

Why It Matters? Jupyter Notebooks promote transparency, reproducibility, and collaboration. Common use cases include:

- Data exploration and pre-processing
- Model training and performance evaluation
- Educational tutorials and teaching resources
- Rapid prototyping in both academia and industry

3 Regression Model

3.1 Linear Regression Model Part 1

Linear regression is a supervised learning algorithm used to fit a straight line to data. It is one of the most widely used methods in machine learning.

Example: Predicting House Prices in Portland

Suppose an estate agent wants to estimate the price of a house based on its size. The available dataset contains historical records of house sizes and their corresponding sale prices.

- The input feature is the size of the house (in square feet).
- The target output is the price (in U.S. dollars).
- Each data point represents a sold house, with both size and price known.

Figure 7 shows a scatter plot of this housing data.

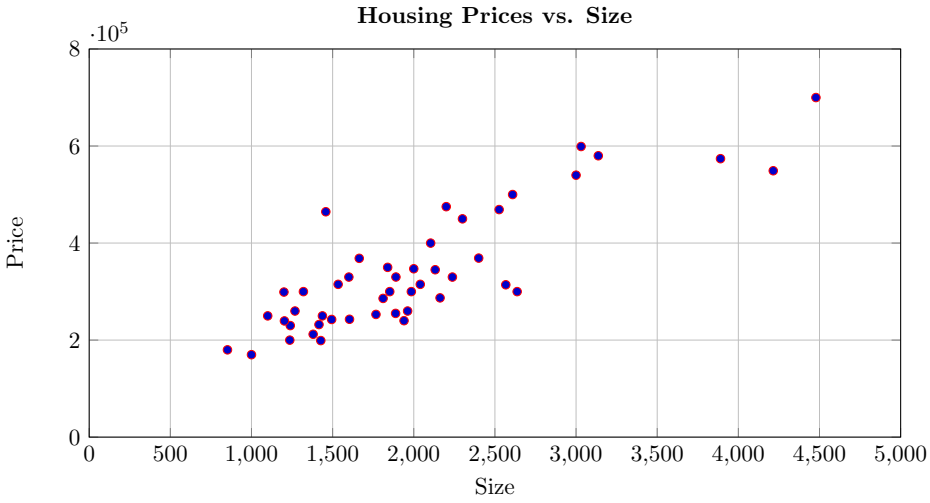


FIGURE 7. Scatter plot of house prices versus house sizes from a real estate dataset.

↪ To make a prediction, a linear regression model is trained on historical data. It learns the relationship between house size and price by fitting a straight line through the data points.

↪ Once the line is fitted, it can be used to estimate the price of a new house. For example, to predict the price of a house that is 1,250 square feet, one would locate 1,250 on the horizontal axis and project upward to the fitted line. The corresponding value on the vertical axis gives the predicted price, as shown in Figure 8.

↪ This is a classic example of supervised learning, where the model is trained on labeled data containing both input values (x) and known outputs (y).

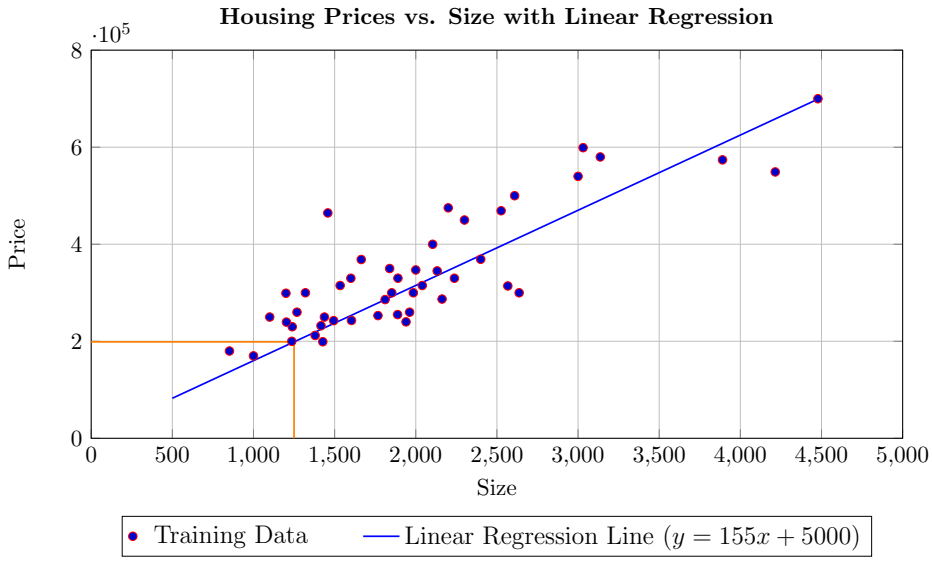


FIGURE 8. Linear regression model fitted to housing data.

In addition to visualizing the data as a plot, it is also possible to view it in tabular form. Both the table and the plot represent the same information, but in different formats.

As shown in Table 1, the first column lists house sizes (in square feet), and the second column shows the corresponding prices (in thousands of dollars). These values correspond to the axes in Figure 8, where size is plotted on the horizontal axis and price on the vertical axis.

Size (ft^2)	**Price ($1000's)**
2104	400
1416	232
1534	315
⋮	⋮

TABLE 1. Tabular representation of the housing dataset.

Standard Notation in Supervised Learning

Machine learning often uses consistent notation to describe data and models. Becoming familiar with this notation will make it easier to understand and communicate machine learning concepts.

The dataset used to train the model is called the **training set**. It contains historical examples, such as house sizes and their corresponding sale prices.

The house whose price is to be predicted (e.g., a client's house) is not included in the training set, since its sale price is not yet known.

The following notation is commonly used:

- x: input variable (also called a *feature*), such as house size
- y: output variable (also called the *target*), such as house price
- (x, y): a training example, consisting of both input and output
- m: total number of training examples in the dataset

In the first training example, $x^{(1)} = 2104$ and $y^{(1)} = 400$, hence $(x^{(1)}, y^{(1)}) = (2104, 400)$. If the dataset contains 47 training examples in total, then $m = 47$.

To refer to a specific example in the training set, the notation $x^{(i)}$ and $y^{(i)}$ is used, where the superscript i in parentheses indicates the index of the training example. For instance, $x^{(1)} = 2104$ and $y^{(1)} = 400$.

3.2 Linear Regression Model Part 2

Training a Model and Making Predictions

Supervised learning models are usually developed using the following steps:

↪ The learning algorithm is provided with a **training set** that contains both input features and corresponding output targets.

↪ Based on this data, the algorithm generates a function f, known as the **model**. (Historically, this was also called a hypothesis.)

↪ The function f takes a new input x and produces a predicted output $\hat{y}$. Note $\hat{y}$ is the model's prediction, while y is the true target value.

Representing the Model Function

A simple way to define the model f is as a linear function:

$$\hat{y} = f_{w,b}(x) = wx + b \tag{1}$$

- w is the weight (slope of the line)

- b is the bias (intercept)
- x is the input feature
- $\hat{y}$ is the predicted output

Sometimes, this function is written simply as $f(x) = wx + b$, omitting the subscripts for clarity when the context is understood.

Why Use a Linear Function?

A linear function (i.e., a straight line) is simple and interpretable. Although more complex, non-linear functions can also be used, linear models serve as a solid foundation and are often surprisingly effective.

- This setup is called **linear regression**, specifically with one input feature. It is also referred to as **univariate linear regression**, where *uni* means one, and *variate* refers to variable.

3.3 Cost Function Formula

To implement linear regression, a crucial first step is to define a **cost function**, which measures how well the model's predictions match the actual outputs. This allows the model to improve by adjusting its parameters.

Model Recap

Given a training dataset with inputs x and corresponding targets y, the model is defined as:

$$\hat{y} = f_{w,b}(x) = wx + b \tag{2}$$

- w is the **weight** or slope of the line.
- b is the **bias** or y-intercept.

These are called the **model parameters**, and they are adjusted during training to improve predictions.

Effect of Parameters w and b

The effect of different values of w and b on the linear function $f_{w,b}(x) = wx + b$ is illustrated in Figure 9 and described below:

- **Example 1: Constant Function**

  Parameters: $w = 0,\ b = 1.5$

  $$f(x) = 0 \cdot x + 1.5 = 1.5 \tag{3}$$

  - Produces a horizontal line at $\hat{y} = 1.5$
  - The output remains constant for any input x

- **Example 2: Line Through the Origin**
  Parameters: $w = 0.5,\ b = 0$

$$f(x) = 0.5x \tag{4}$$

  – A diagonal line with slope 0.5
  – Passes through the origin $(0, 0)$

- **Example 3: Diagonal Line with Intercept**
  Parameters: $w = 0.5,\ b = 1$

$$f(x) = 0.5x + 1 \tag{5}$$

  – Same slope as Example 2 ($w = 0.5$)
  – Intercepts the y-axis at $b = 1$

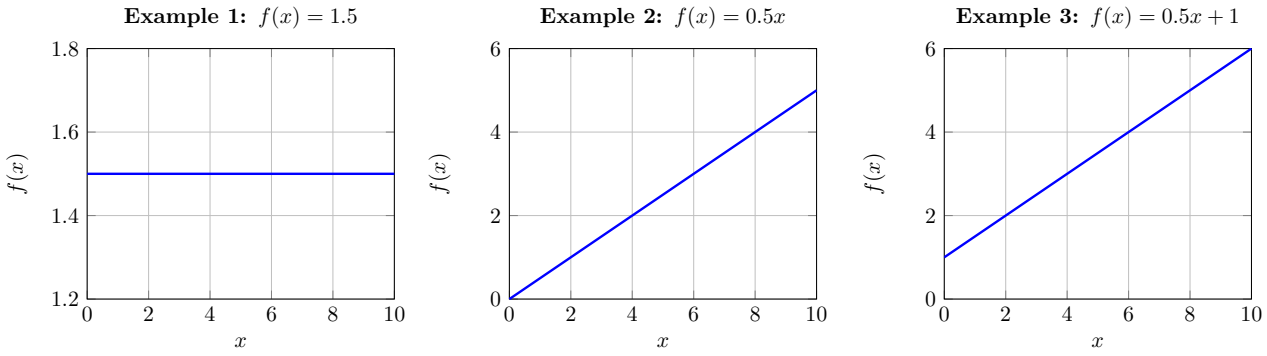


FIGURE 9. Effect of varying w and b on the linear function $f(x) = wx + b$.

Model Predictions

The objective in training a linear regression model is to find values for the parameters w and b such that the prediction function $\hat{y}^{(i)} = f_{w,b}(x^{(i)}) = wx^{(i)} + b$ produces outputs $\hat{y}^{(i)}$ that closely match the actual target values $y^{(i)}$ for all training examples $(x^{(i)}, y^{(i)})$.

Constructing the Cost Function

(1) Prediction for the i-th training example $\hat{y}^{(i)} = wx^{(i)} + b$

(2) Compute the error $\text{Error}^{(i)} = \hat{y}^{(i)} - y^{(i)}$

(3) Square the error to penalize larger mistakes $\left(\hat{y}^{(i)} - y^{(i)}\right)^2$

(4) Sum over all training examples $\sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right)^2$

(5) Compute the average error $\frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right)^2$

(6) Final cost function (for optimization purposes) $J(w, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)}\right)^2$

This cost function is known as the **Mean Squared Error (MSE)** and is widely used in regression problems.

Interpretation:

- A large $J(w, b)$ means predictions are far from true values.
- A small $J(w, b)$ means the model fits the data well.

3.4 Cost Function Intuition

Recap: The Model and the Cost Function

In linear regression, the model is defined as:

$$f_{w,b}(x) = wx + b \tag{6}$$

where w is the weight controlling the slope, and b is the bias controlling the intercept.

To quantify the model's performance, a cost function is defined as:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2 \tag{7}$$

The objective of training is to determine values of w and b that minimize this cost:

$$\min_{w,b} J(w, b) \tag{8}$$

Simplified Model for Visualization

To facilitate graphical interpretation, a simplified version of the model is considered by setting $b = 0$, resulting in:

$$f_w(x) = wx \tag{9}$$

The associated cost function becomes:

$$J(w) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_w(x^{(i)}) - y^{(i)} \right)^2 \tag{10}$$

Illustrative Examples

A simple dataset with three training examples is used to illustrate the cost function:

$$(1, 1), \quad (2, 2), \quad (3, 3)$$

- **Case 1:** $w = 1$

$$f(1) = 1.0, \;\; \rightarrow \;\; \text{error} = (1-1)^2 = 0.0$$

$$f(2) = 2.0, \;\; \rightarrow \;\; \text{error} = (2-2)^2 = 0.0$$

$$f(3) = 3.0, \;\; \rightarrow \;\; \text{error} = (3-3)^2 = 0.0$$

$$J(1) = \frac{1}{2(3)} \cdot (0.0 + 0.0 + 0.0) = 0.0$$

- **Case 2:** $w = 0.5$

$$f(1) = 0.5, \;\; \rightarrow \;\; \text{error} = (0.5-1)^2 = 0.25$$

$$f(2) = 1.0, \;\; \rightarrow \;\; \text{error} = (1.0-2)^2 = 1.00$$

$$f(3) = 1.5, \;\; \rightarrow \;\; \text{error} = (1.5-3)^2 = 2.25$$

$$J(0.5) = \frac{1}{2(3)} \cdot (0.25 + 1.00 + 2.25) \approx 0.58$$

- **Case 3:** $w = 0$

$$f(1) = 0.0, \;\; \rightarrow \;\; \text{error} = (0.0-1)^2 = 1.0$$

$$f(2) = 0.0, \;\; \rightarrow \;\; \text{error} = (0.0-2)^2 = 4.0$$

$$f(3) = 0.0, \;\; \rightarrow \;\; \text{error} = (0.0-3)^2 = 9.0$$

$$J(0.5) = \frac{1}{2(3)} \cdot (1.0 + 4.0 + 9.0) \approx 2.33$$

- **Case 4:** $w = -0.5$

$$f(1) = -0.5, \;\; \rightarrow \;\; \text{error} = (-0.5-1)^2 = 2.25$$

$$f(2) = -1.0, \;\; \rightarrow \;\; \text{error} = (-1.0-2)^2 = 9.00$$

$$f(3) = -1.5, \;\; \rightarrow \;\; \text{error} = (-1.5-3)^2 = 20.25$$

$$J(-0.5) = \frac{1}{2(3)} \cdot (2.25 + 9.00 + 20.25) \approx 5.25$$

Figure 10 shows the considered cases and illustrates how different values of the weight w affect the model's predictions.

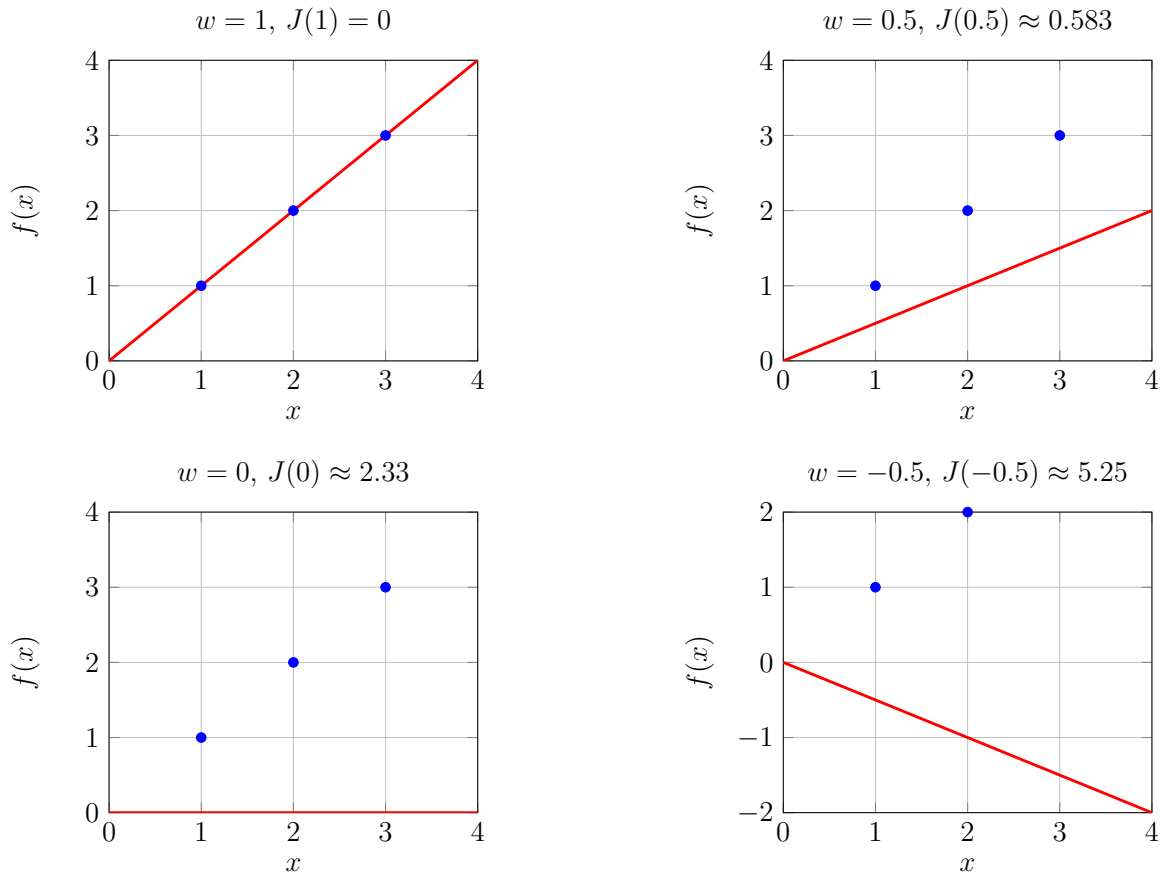


FIGURE 10. Prediction lines for various weights on the dataset $(1, 1), (2, 2), (3, 3)$.

Evaluating $J(w)$ over a range of w results in a U-shaped curve (see Figure 11).

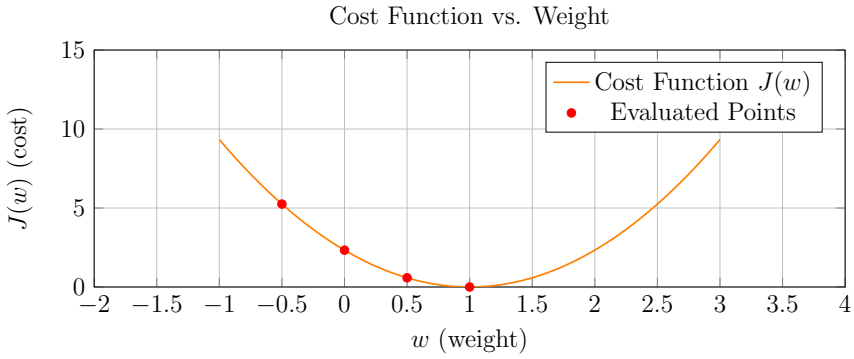


FIGURE 11. Cost function values for different weight choices w.

Figure 11 shows how the model's prediction error changes with different choices of w. The lowest point on the curve corresponds to the best-performing model. In this case, the minimum occurs at $w = 1$ with $J(1) = 0$.

3.5 Visualizing The Cost Function

In earlier visualizations, b was temporarily set to zero to simplify the analysis. Now, both w and b will be considered to understand the full geometry of the cost function.

3D Surface Plot

- ↪ When both w and b change, the cost becomes a surface over the (w, b)-plane.
  - ↪ Each point (w, b) has a height $J(w, b)$ that measures the model's error for that parameter choice.
    - ↪ For quadratic cost, the surface is bowl-shaped; moving away from the minimum increases $J(w, b)$.
    - ↪ The lowest point on the surface gives the optimal w and b, where the error is minimized.

Example: Let $w = 0.06$ and $b = 50$. In this case, the linear model becomes:

$$f(x) = 0.06x + 50 \tag{11}$$

This model tends to predict prices lower than the actual values, leading to a relatively high cost $J(w, b)$, which reflects a poor fit to the data (see Figure 12).

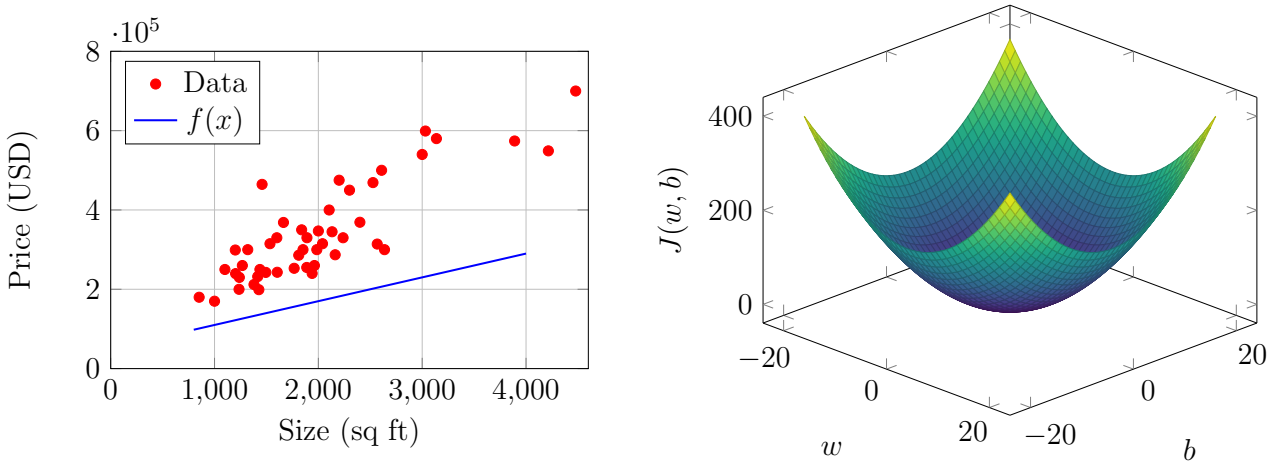


FIGURE 12. Visualization of a linear regression model and its associated cost function surface.

Contour Plots: A 2D View of a 3D Surface

- A **contour plot** is a 2D representation of a 3D surface that helps simplify the visualisation of a cost function (see Figure 13).
  - It shows horizontal slices of the 3D surface and illustrates how the cost value $J(w, b)$ varies for different combinations of parameters w and b.
  - Each closed curve (ellipse or oval) in the plot is a *level set*, connecting all points with the same cost value.
  - The innermost contour represents the lowest cost and indicates the optimal parameter values.

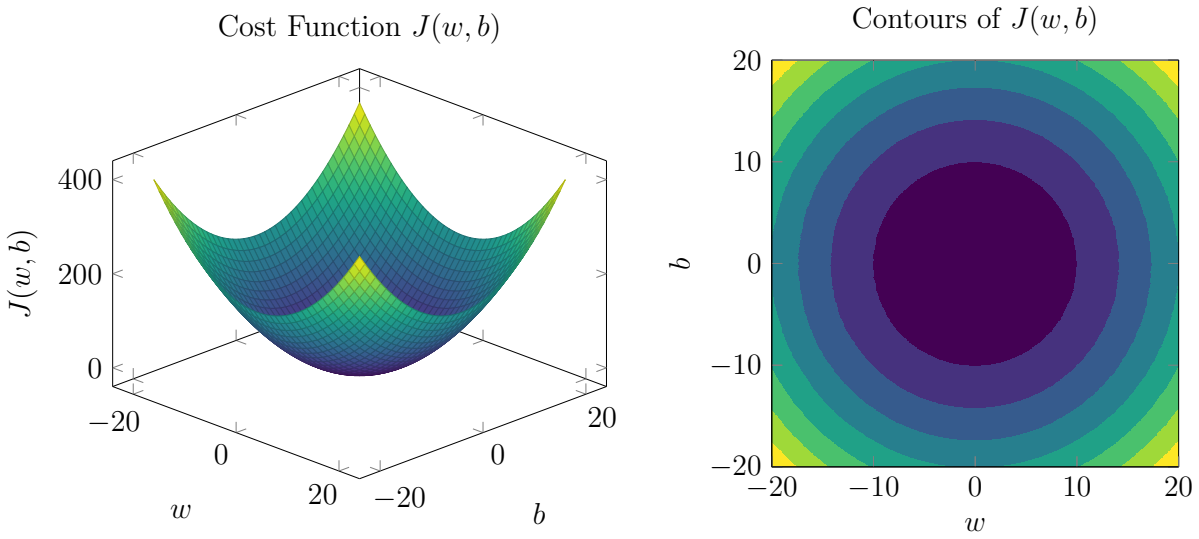


FIGURE 13. Visualisation of the cost function $J(w, b)$ in 3D and 2D contour views.

3.6 Visualization Examples

The following examples illustrate how different values of the parameters w (weight) and b (bias) influence the linear model's predictions and the resulting cost $J(w, b)$.

Example 1: $w = -0.15$, $b = 800$

Function: $f(x) = -0.15x + 800$

Effect:

- The line has a negative slope and a high intercept. It poorly fits the training data.
- Predictions are far from the actual target values.

Cost: High; the corresponding point on the cost surface is far from the minimum.

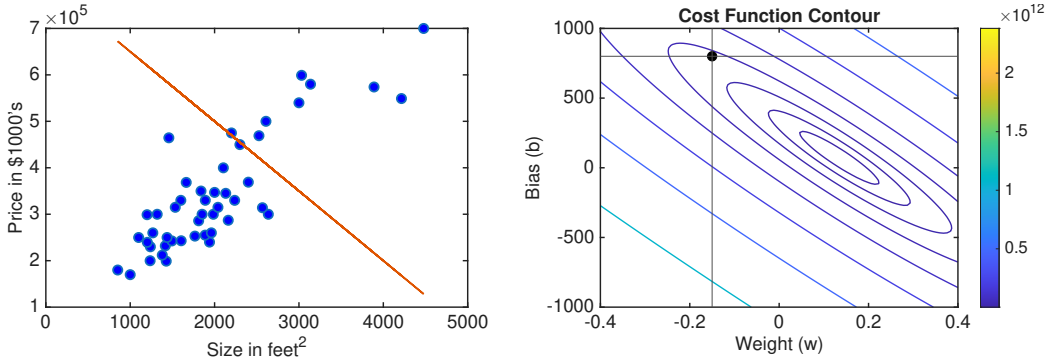


FIGURE 14. Contour of $J(w, b)$ with optimum $(-0.15, 800)$ in black.

Example 2: $w = 0$, $b = 360$

Function: $f(x) = 360$

Effect:

- The model outputs a constant value for all inputs.
- Still a poor fit but marginally closer to the true values than Example 1.

Cost: Lower than in Example 1, but still suboptimal.

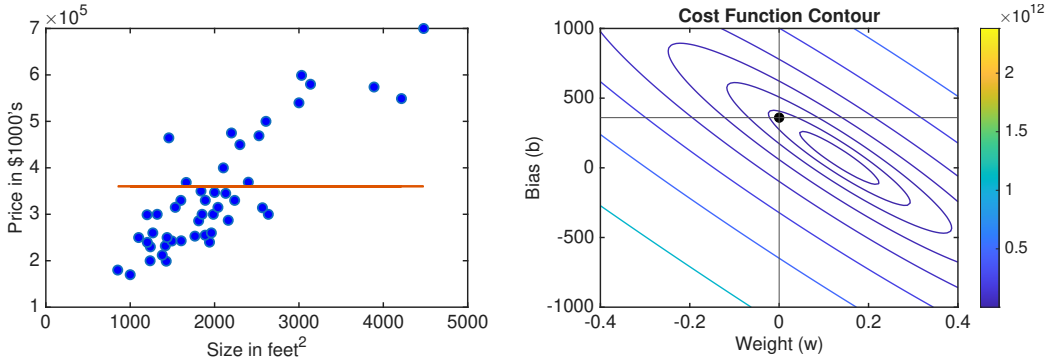


FIGURE 15. Contour of $J(w, b)$ with optimum $(0, 360)$ in black.

Example 3: $w = -0.15$, $b = 500$

Function: $f(x) = -0.15x + 500$

Effect:

- This fit is worse than Example 2.
- Predictions are again farther from targets.

Cost: Higher again; further from the centre of the cost contour.

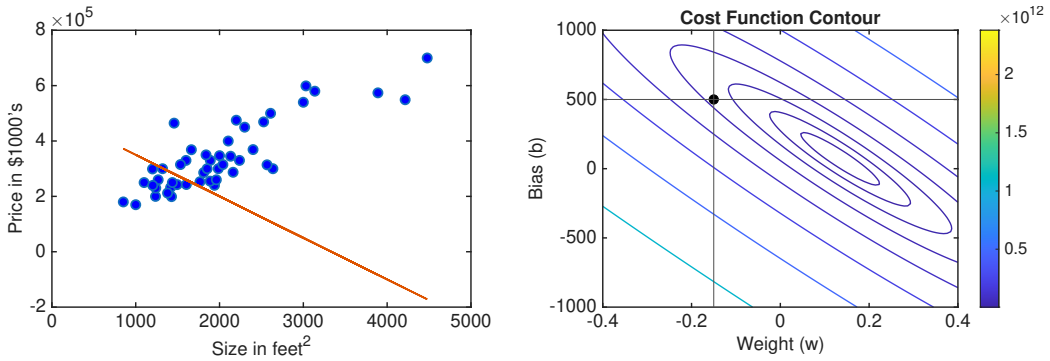


FIGURE 16. Contour of $J(w, b)$ with optimum $(-0.15, 500)$ in black.

Example 4: $w = 0.13$, $b = 71$

Function: $f(x) = 0.13x + 71$

Effect:

- The line fits the data well.
- Prediction errors are small.

Cost: Close to the minimum on the cost surface.

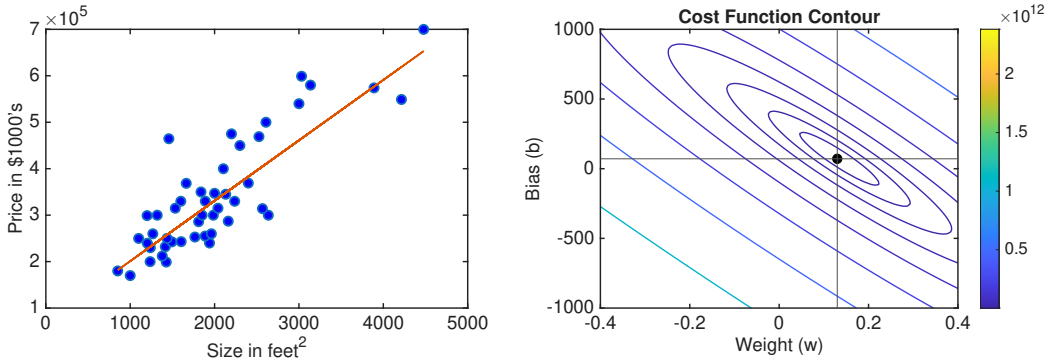


FIGURE 17. Cost contour of $J(w, b)$ with optimum $(0.13, 71)$ in black.

Why Manual Tuning Fails

- Manually selecting w and b is inefficient and not scalable.
- For complex models and large datasets, a better solution is required.

Gradient Descent

- Gradient descent is an algorithm that finds optimal values of w and b by minimising $J(w, b)$. It is widely used in machine learning, from linear models to deep neural networks.

4 Train The Model With Gradient Descent

4.1 Gradient Descent

In previous discussions, the cost function $J(w, b)$ was visualized to illustrate how different parameter choices influence model performance. Rather than testing values randomly, it is more effective to apply a systematic method to identify the values of w and b that minimize the cost.

What Is Gradient Descent? Gradient descent is an iterative algorithm designed to minimise a function by repeatedly adjusting parameters in the direction that most rapidly decreases the function's value.

How Gradient Descent Works

(1) **Initialize Parameters** Start with initial guesses for the parameters. A common choice is $w = 0$ and $b = 0$.

(2) **Iterative Update** Gradually adjust w and b in the direction that most rapidly decreases the cost function. This direction is given by the negative of the gradient of $J(w, b)$.

(3) **Repeat** Continue updating until the cost function converges to a minimum value.

Local vs Global Minima

- **Local minimum:** A local minimum is a point where the cost is lower than nearby points, but it might not be the lowest value possible across the whole function.
- **Global minimum:** The global minimum is the point where the cost function reaches its lowest possible value over the entire parameter space.

Local Minima and Initialization Sensitivity

For simple cost functions like those in linear regression with squared error, the cost surface forms a convex bowl, ensuring a unique global minimum.

However, for more complex functions such as those used in training neural networks the cost surface may contain multiple local minima. Starting from different initial positions can lead to convergence at different minima.

4.2 Implementing Gradient Descent

General Update Form

To minimize the cost function $J(w, b)$, gradient descent iteratively updates the model parameters w and b using the following rules

$$w := w - \alpha \frac{\partial J(w, b)}{\partial w}, \qquad b := b - \alpha \frac{\partial J(w, b)}{\partial b} \tag{12}$$

In this formula

- α is the **learning rate**, a hyperparameter that controls the step size taken in the direction of the negative gradient at each iteration.

  - If α is too small, gradient descent will move very slowly toward the minimum, taking many steps to converge.
  - If α is too large, the updates may overshoot the minimum or cause the algorithm to diverge entirely.
  - A typical learning rate falls in the range $0 < \alpha < 1$, with common values like 0.01 or 0.001.

- The term $\frac{\partial}{\partial w} J(w, b)$ represents the partial derivative of the cost function with respect to w. It tells us how sensitive the cost is to changes in w.

- Similarly, $\frac{\partial}{\partial b} J(w, b)$ is the partial derivative with respect to b, indicating how the cost changes as b changes.

Programming Note: Assignment vs Equality

In programming, the assignment operator `=` is used to update a variable's value. For example:

```
w = w - alpha * derivative
```

- This means: Take the current value of `w`, subtract the product of `alpha` and the derivative, and assign the result back to `w`.
- This is different from mathematical equality, which in many programming languages is expressed using `==` and simply checks whether two values are the same.

Simultaneous vs Sequential Parameter Updates

When updating multiple parameters like w and b, it is important to compute their new values based on the same original state.

Correct (Simultaneous Update): First compute the updated values without changing w or b immediately:

```
temp_w = w - alpha * dJ_dw
temp_b = b - alpha * dJ_db
w = temp_w
b = temp_b
```

Incorrect (Sequential Update): If you update w before computing the update for b, then b's update is based on a modified value of w, which breaks the gradient descent logic:

```
temp_w = w - alpha * dJ_dw
w = temp_w                    # w is now updated
temp_b = b - alpha * dJ_db    # uses updated w
b = temp_b
```

This violates the principle of using the same snapshot of parameters for both updates, leading to incorrect behavior in the algorithm.

4.3 Gradient Descent Intuition

Gradient descent is an algorithm used to minimize the cost function $J(w, b)$ by iteratively adjusting the model parameters w and b.

Gradient Descent Update Rule For a single parameter w, the update rule is:

$$w := w - \alpha \frac{d}{dw} J(w) \tag{13}$$

where

- α is the **learning rate**, which controls the size of the update step.
- $\frac{d}{dw} J(w)$ is the **derivative** (or technically, the *partial derivative*) of the cost function with respect to w, indicating the slope or direction of change.

Intuition Using a 1D Cost Function

To build intuition, consider a cost function $J(w)$ that depends on a single parameter w. The horizontal axis represents w, and the vertical axis represents $J(w)$.

Case 1: Suppose the current value of w is to the right of the minimum.

- The slope of the tangent line is **positive**.
- Therefore, the derivative $\frac{d}{dw} J(w) > 0$.
- The update becomes: $w := w - \alpha \times$ (positive), which decreases w.
- This moves the parameter left (toward the minimum).

Case 2: Suppose the current value of w is to the left of the minimum.

- The slope is **negative**, so $\frac{d}{dw} J(w) < 0$.
- The update becomes: $w := w - \alpha \times$ (negative), which increases w.
- This moves the parameter right (toward the minimum).

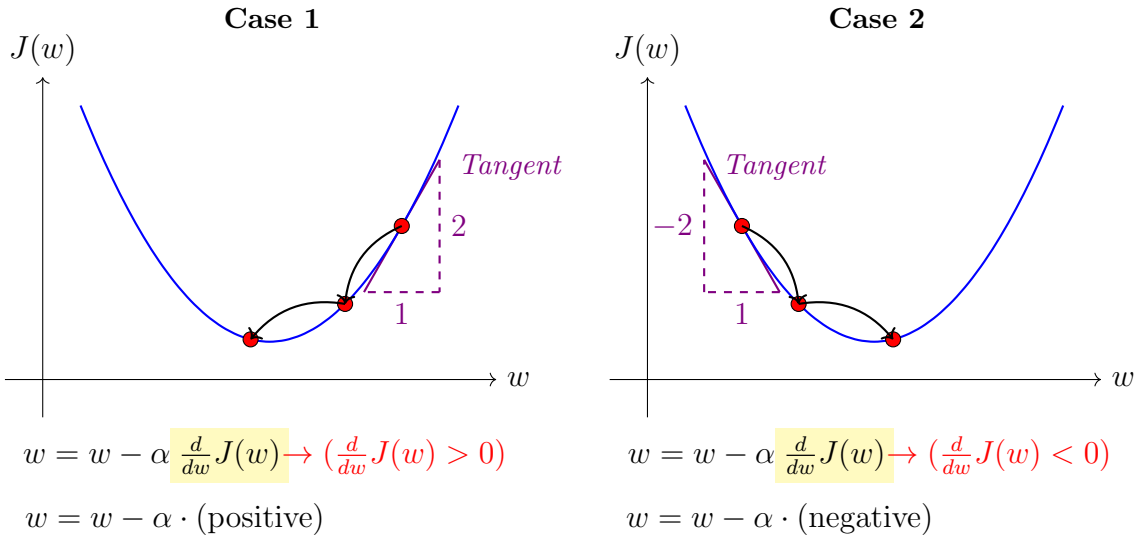


FIGURE 18. Gradient descent updates w based on the slope of $J(w)$: move left if slope is positive, right if negative.

4.4 Learning Rate

The learning rate α plays a critical role in determining the efficiency of gradient descent. A poorly chosen α can lead to slow convergence or even prevent convergence altogether.

Gradient Descent Update Rule

$$w := w - \alpha \frac{d}{dw} J(w) \tag{14}$$

This rule updates the parameter w using the gradient of the cost function $J(w)$ and the learning rate α.

- When α is very small (e.g., 10^{-6}), updates to w are minimal.
  - This results in slow progress: gradient descent takes many tiny steps toward the minimum.
  - Although the algorithm eventually converges, it does so very inefficiently.
- If α is too large, updates overshoot the minimum.
  - The algorithm may jump back and forth across the cost curve.
  - In extreme cases, the cost increases with each step, and the algorithm diverges.
  - Gradient descent fails to converge.

(a) Small learning rate (b) Large learning rate (c) Stable learning rate

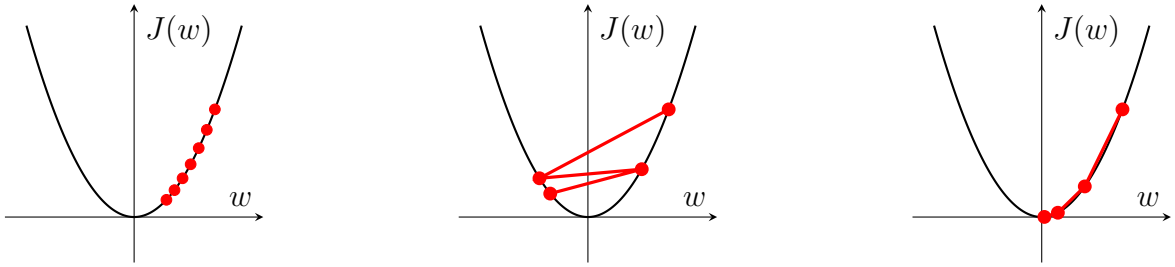


FIGURE 19. Effect of learning rate α on gradient descent. Left: a small α results in slow convergence. Middle: a large α leads to divergence or oscillation. Right: a well-chosen α enables efficient convergence.

Behavior at a Local Minimum

- At a local minimum, the gradient $\frac{d}{dw}J(w) = 0$.
- The update rule becomes: $w := w - \alpha \cdot 0 = w$, so the parameter stays the same.
- Gradient descent halts at this point, as desired.

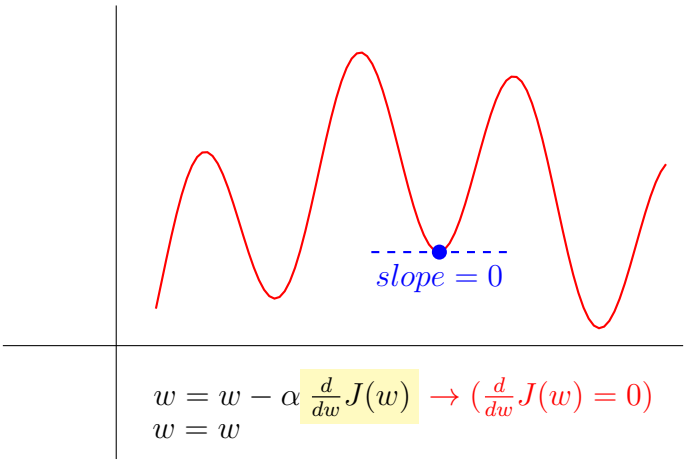


FIGURE 20. Gradient descent reaches a local minimum when the slope becomes zero, resulting in no further parameter updates.

Why Gradient Descent Takes Smaller Steps Near a Minimum

- Early in training, gradients are large, resulting in larger steps.
- As the algorithm approaches a minimum, gradients decrease.
- This naturally leads to smaller updates, even if α remains fixed.

4.5 Gradient Descent For Linear Regression

Previously, the linear regression model, cost function, and gradient descent algorithm were introduced. This section shows how to apply gradient descent to minimize the squared error cost function.

- **Model and Cost Function**
  - Linear regression model

$$f_{w,b}(x^{(i)}) = wx^{(i)} + b \tag{15}$$

  - Cost function

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)^2 \tag{16}$$

- **Gradient Descent Algorithm**
  - Update rules

$$w := w - \alpha \cdot \frac{\partial J(w,b)}{\partial w}, \qquad b := b - \alpha \cdot \frac{\partial J(w,b)}{\partial b} \tag{17}$$

  - Partial derivatives

$$\frac{\partial J(w,b)}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)x^{(i)}, \qquad \frac{\partial J(w,b)}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right) \tag{18}$$

Convergence and Convexity

↪ When using the squared error cost function in linear regression, the cost function $J(w,b)$ is **convex**.

↪ A convex function has a single **global minimum** and no other local minima.

↪ This means gradient descent will always converge to the global minimum as long as the learning rate α is chosen appropriately.

Module 2: Regression with Multiple Input Variables

1 Multiple Linear Regression

1.1 Multiple Features

In the basic form of linear regression, the model uses a single input feature, denoted by x. For example, x could represent the *size of a house*, and the corresponding output y would be its price. The model predicts y using a linear relationship:

$$f_{w,b}(x) = wx + b \tag{19}$$

Now consider a case where multiple features are available to predict house prices. For instance, in addition to the size of the house, one might also include the following:

- x_1: size of the house (in square feet)
- x_2: number of bedrooms
- x_3: number of floors
- x_4: age of the house (in years)

Each input feature is identified by a subscript j, with the j-th feature written as x_j. The total number of features is denoted by n; in this example, $n = 4$.

Notation

- $x^{(i)}$ denotes the feature vector for the i-th training example.
- $x_j^{(i)}$ refers to the j-th feature of the i-th training example.

Table 2 displays a sample training dataset with multiple input features. The highlighted second row corresponds to the training example indexed by $i = 2$, with the following feature vector and target value:

$$\mathbf{x}^{(2)} = \begin{bmatrix} 1416 & 3 & 2 & 40 \end{bmatrix}, \quad y^{(2)} = 232. \tag{20}$$

The third element of the feature vector, indicating the number of floors, is given by $x_3^{(2)} = 2$.

Model with Multiple Features

In the case of multiple input features, the linear regression model is expressed as:

$$f_{w,b}(x) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b \tag{21}$$

Size (x_1)	**Bedrooms** (x_2)	**Floors** (x_3)	**Age** (x_4)	**Price**
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
⋮	⋮	⋮	⋮	⋮

TABLE 2. Sample training set with multiple features

where $x_1, x_2, \ldots, x_n$ denote the input features, $w_1, w_2, \ldots, w_n$ are their associated weights (parameters) that determine the influence of each feature, and b is the bias term.

Example: Consider a model with $n = 4$ input features and the following parameter values:

$$w_1 = 0.1, \qquad w_2 = 4, \qquad w_3 = 10, \qquad w_4 = -2, \qquad b = 80 \tag{22}$$

The resulting model is:

$$f_{w,b}(x) = 0.1x_1 + 4x_2 + 10x_3 - 2x_4 + 80 \tag{23}$$

This form illustrates how each input feature influences the final prediction: each feature x_j is multiplied by its corresponding weight w_j, and the result is adjusted by the bias term b.

Vector Notation

To express the model in a more concise way, the individual parameters and input features can be grouped into vectors. Vectors may be denoted either with an arrow above the symbol (e.g., $\vec{w}$, $\vec{x}$) or in boldface (e.g., $\mathbf{w}$, $\mathbf{x}$). The arrow notation is used in the expressions below:

$$\vec{w} = \begin{bmatrix} w_1 & w_2 & w_3 & \ldots & w_n \end{bmatrix}, \qquad \vec{x} = \begin{bmatrix} x_1 & x_2 & x_3 & \ldots & x_n \end{bmatrix} \tag{24}$$

The model can now be written compactly as:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b \tag{25}$$

Dot Product

The dot product of two vectors $\vec{w}$ and $\vec{x}$ produces a scalar (a single number). It is computed as the sum of the products of their corresponding components:

$$\vec{w} \cdot \vec{x} = w_1x_1 + w_2x_2 + \cdots + w_nx_n = \sum_{j=1}^{n} w_jx_j \tag{26}$$

This scalar result is then combined with the bias term b to define the linear model:

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b = \sum_{j=1}^{n} w_j x_j + b \tag{27}$$

Terminology

- **Multiple linear regression**: a linear model that uses more than one input feature (independent variable).
- **Univariate linear regression**: a linear model that uses only a single input feature.

1.2 Vectorization Part 1

Vectorization is a key concept in machine learning that allows models to be implemented in a way that is both **concise** and **efficient**. By representing data and operations as vectors and matrices, algorithms can take advantage of highly optimized linear algebra libraries and hardware acceleration (such as GPUs), leading to significantly faster computations.

Example: Consider a simple linear model with weight vector $\vec{w}$, input vector $\vec{x}$, and bias b:

$$\vec{w} = \begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix}, \quad \vec{x} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \tag{28}$$

The prediction of the model is computed as the dot product of $\vec{w}$ and $\vec{x}$, followed by the addition of the bias term.

$$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b \tag{29}$$

The dot product $\vec{w} \cdot \vec{x}$ yields a single scalar value:

$$\vec{w} \cdot \vec{x} = w_1 x_1 + w_2 x_2 + w_3 x_3 = \sum_{j=1}^{3} w_j x_j \tag{30}$$

The same calculation can be implemented in Python using the `NumPy` library. `NumPy` (short for *Numerical Python*) is a core library for numerical computing in Python. It provides support for arrays and efficient mathematical operations, such as the dot product.

```
import numpy as np
w = np.array([1.2, 3.4, 5.6])       # weight vector
x = np.array([7.8, 9.0, 1.2])       # input vector
b = 0.5                             # bias term
f = np.dot(w, x) + b                # prediction
```

The line `np.dot(w, x)` computes the dot product $\vec{w} \cdot \vec{x}$, and adding `b` completes the prediction. This approach is concise and takes advantage of highly optimized, low-level numerical libraries for fast computation.

The same computation can be written without vectorization using an explicit loop:

```
f = 0
for j in range(3):
    f += w[j] * x[j]            # manually compute dot product
f += b                          # add bias
```

This loop-based approach works for small input sizes and is easy to understand. However, it becomes inefficient for models with hundreds or thousands of features, as each operation is performed sequentially in Python rather than in optimized native code.

Why Vectorization is Faster

The function `np.dot` used in the vectorized implementation is backed by low-level linear algebra libraries such as BLAS. These libraries take advantage of parallel hardware (including CPUs and GPUs) and are highly optimized for performance. While a loop processes each multiplication and addition one at a time, vectorized operations perform all calculations in bulk using fast compiled routines.

Benefits of Vectorization

- Concise Code: Easier to write, debug, and maintain.
- Scalability: Handles large datasets and high-dimensional models efficiently.
- Faster Execution: Leverages parallel hardware and optimized numerical libraries.

1.3 Vectorization Part 2

A vectorized algorithm often runs much faster than its non-vectorized counterpart, and the performance improvement can seem almost magical at first. To understand why this happens, consider how a non-vectorized loop operates. Suppose the loop runs over indices $j = 0, 1, \ldots, 15$. In this setup, each computation happens one step at a time:

- At time t_0, the computation uses $j = 0$
- At time t_1, the computation uses $j = 1$
- $\ldots$
- At time t_{15}, the computation uses $j = 15$

Each iteration must wait for the previous one to finish. This is called **sequential execution** and is common in standard Python loops.

By contrast, libraries like `NumPy` take advantage of **vectorization**, which allows operations to run in parallel. When computing something like the dot product between two vectors $\vec{w}$ and $\vec{x}$, the process works very differently:

(1) All element-wise multiplications $w_j \cdot x_j$ are performed simultaneously.

(2) The results are then summed using fast, hardware-optimized routines.

The difference between non-vectorized and vectorized computation is illustrated in Figure 21. In the non-vectorized version, each element is processed sequentially in a loop. In the vectorized version, operations are executed in parallel, making computation significantly faster.

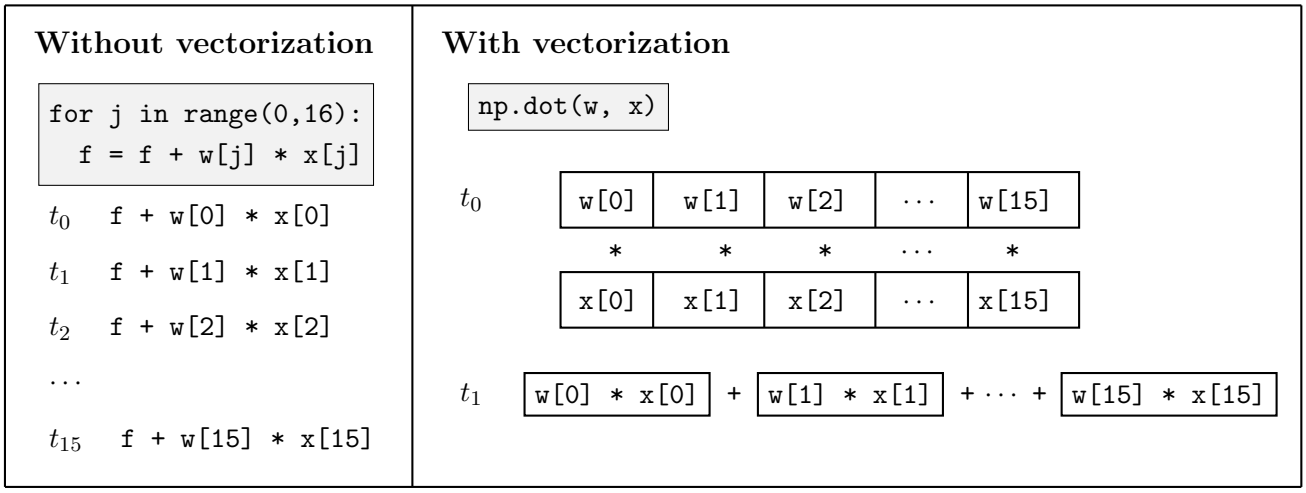


FIGURE 21. Vectorized vs non-vectorized dot product computation.

1.4 Gradient Descent For Multiple Linear Regression

Gradient descent can be used to train a linear regression model that works with multiple input features. Vectorization makes this process faster and more efficient. To measure how well the model performs, a cost function is used. For m training examples, the cost is:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right)^2 \tag{31}$$

The goal is to find $\vec{w}$ and b that minimize this cost.

Gradient descent is an iterative method that updates the parameters to reduce the cost. The update rules for each weight and the bias are:

$$w_j := w_j - \alpha \frac{\partial J(\vec{w}, b)}{\partial w_j}, \quad \text{for each } j = 1, 2, \ldots, n \qquad \text{and} \qquad b := b - \alpha \frac{\partial J(\vec{w}, b)}{\partial b} \tag{32}$$

The learning rate α controls how big each update step is.

The partial derivatives used in the updates are:

$$\frac{\partial J(\vec{w}, b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} \tag{33}$$

$$\frac{\partial J(\vec{w}, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \left(f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)} \right) \tag{34}$$

These formulas calculate how much each parameter contributes to the prediction error.

With vectorized operations, the update to all weights can be written more compactly as:

$$\vec{w} := \vec{w} - \alpha \cdot \nabla_{\vec{w}} J \quad \text{and} \quad b := b - \alpha \cdot \frac{\partial J}{\partial b} \tag{35}$$

Vectorization avoids loops and uses fast matrix operations, which is especially useful when training on large datasets.

Alternative: The Normal Equation

Another method for solving linear regression is the *normal equation*, which finds the best parameters directly using a closed-form formula.

While the normal equation does not require iterative updates, it has some drawbacks:

- It only works for linear regression.
- It becomes slow and memory-intensive when the number of features is large.

Some machine learning libraries use the normal equation when the dataset is small, but gradient descent is generally more practical for large or complex problems.

2 Gradient Descent In Practice

2.1 Feature Scaling Part 1

Gradient descent can work much more efficiently when using a technique called **feature scaling**. This is especially helpful when the input features have very different numeric ranges.

Consider a model that predicts the price of a house using two input features:

- x_1: Size of the house in square feet, typically ranging from 300 to 2000
- x_2: Number of bedrooms, typically ranging from 0 to 5

Assume a training example with the following values:

$$x_1 = 2000, \quad x_2 = 5, \quad y = 500 \quad \text{(representing \$500,000)} \tag{36}$$

The following two sets of parameter values demonstrate how the scale of input features influences the corresponding weights.

Example 1:

$$w_1 = 50, \quad w_2 = 0.1, \quad b = 50 \tag{37}$$

$$f(\vec{x}) = 50 \cdot 2000 + 0.1 \cdot 5 + 50 = 100000 + 0.5 + 50 = 100050.5 \tag{38}$$

The result significantly overpredicts the target value of 500 (i.e., \$500,000).

Example 2:

$$w_1 = 0.1, \quad w_2 = 50, \quad b = 50 \tag{39}$$

$$f(\vec{x}) = 0.1 \cdot 2000 + 50 \cdot 5 + 50 = 200 + 250 + 50 = 500 \tag{40}$$

This prediction matches the actual output and uses more balanced weight values.

These examples highlight a common pattern: features with large numeric ranges (e.g., square footage) tend to require smaller weights, while features with smaller ranges (e.g., number of bedrooms) typically require larger weights to exert similar influence.

Impact on Gradient Descent

Large differences in the scale of input features can distort the shape of the cost function. This causes the contour lines of the cost surface to become stretched or elongated, which makes gradient descent inefficient. For example:

- A small change in w_1, linked to a feature with a large range (e.g., house size), leads to a large change in the model's prediction.
- A much larger change in w_2, associated with a smaller-range feature (e.g., number of bedrooms), is needed to produce the same effect.

As a result, gradient descent zigzags across the cost surface and converges slowly.

Figure 22 illustrates this effect:

- **Left column:** Before scaling (top), features have different numeric ranges. After scaling (bottom), they are adjusted to similar ranges.
- **Right column:** Without scaling (top), the cost contours are elongated and descent is inefficient. After scaling (bottom), the contours are circular, and gradient descent converges faster and more directly.

Solution: Feature Scaling

To fix this imbalance, input features can be rescaled so they are on similar numerical scales for example, between 0 and 1, or by standardizing them to have a mean of 0 and a standard deviation of 1.

- This makes the cost function more balanced, with contour lines that are more circular rather than stretched.
- As a result, gradient descent can move more directly toward the minimum, improving convergence speed.

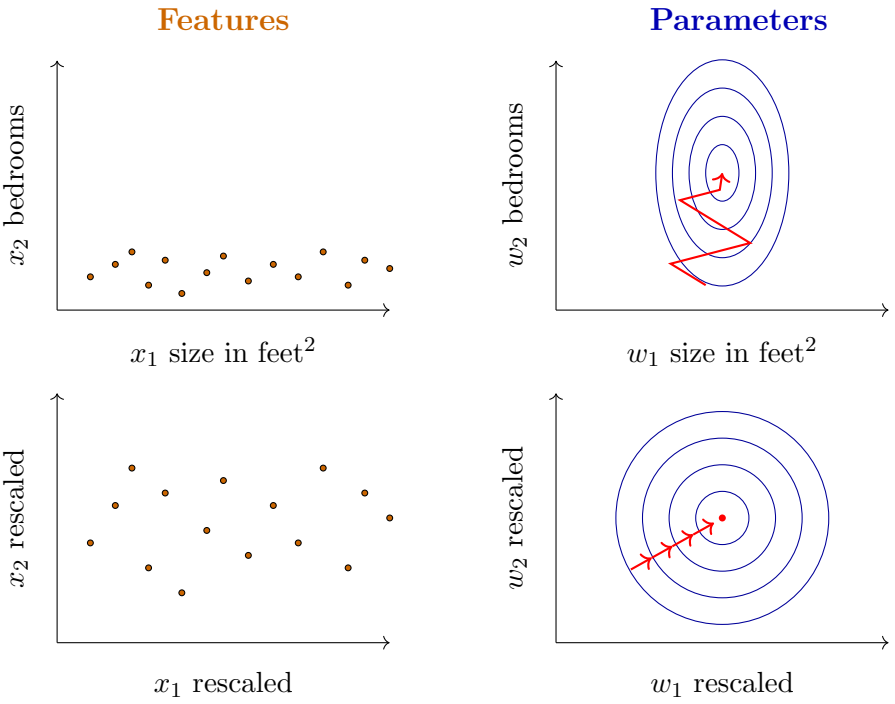


FIGURE 22. Effect of feature scaling on optimization dynamics.

2.2 Feature Scaling Part 2

Feature scaling helps gradient descent perform more efficiently by ensuring that all input features are on similar numeric scales. When features vary widely in scale, the optimization process can become unstable or slow to converge. Below are the most common techniques used to scale features before training a machine learning model.

1. Min-Max Scaling

Min-max scaling is a method that adjusts a feature so that all of its values fall between 0 and 1. It does this by dividing each value by the maximum value of that feature:

$$x_j^{(\text{scaled})} = \frac{x_j}{\max(x_j)} \tag{41}$$

Example: Suppose a feature x_j has values ranging from 0 to 100. Dividing each value by 100 gives:

$$x_j^{(\text{scaled})} = \frac{x_j}{100} \tag{42}$$

This transforms all values into the range $[0, 1]$.

Additional examples:

- If x_1 (e.g., house size) ranges from 300 to 2000:

$$x_1^{(\text{scaled})} = \frac{x_1}{2000} \quad \Rightarrow \quad x_1^{(\text{scaled})} \in [0.15, 1] \tag{43}$$

- If x_2 (e.g., number of bedrooms) ranges from 0 to 5:

$$x_2^{(\text{scaled})} = \frac{x_2}{5} \quad \Rightarrow \quad x_2^{(\text{scaled})} \in [0, 1] \tag{44}$$

2. Mean Normalization

Mean normalization adjusts feature values so they are centered around zero and scaled to lie roughly between -1 and 1. The formula for mean normalization is:

$$x_j^{(\text{norm})} = \frac{x_j - \mu_j}{\max(x_j) - \min(x_j)} \tag{45}$$

Example: Suppose the feature x_j has a mean value μ_j, and known minimum and maximum values. You subtract the mean and divide by the range (max - min).

- If x_1 (e.g., house size) has:

$$\mu_1 = 600, \quad \max(x_1) = 2000, \quad \min(x_1) = 300 \tag{46}$$

  Then the normalized version of x_1 is:

$$x_1^{(\text{norm})} = \frac{x_1 - 600}{1700} \quad \Rightarrow \quad x_1^{(\text{norm})} \in [-0.18, 0.82] \tag{47}$$

- If x_2 (e.g., number of bedrooms) has:

$$\mu_2 = 2.3, \quad \max(x_2) = 5, \quad \min(x_2) = 0 \tag{48}$$

  Then:

$$x_2^{(\text{norm})} = \frac{x_2 - 2.3}{5} \quad \Rightarrow \quad x_2^{(\text{norm})} \in [-0.46, 0.54] \tag{49}$$

3. Z-score Normalization

Also called standardization, this method rescales a feature so that it has a mean of 0 and a standard deviation of 1. The formula for Z-score normalization is:

$$x_j^{(z)} = \frac{x_j - \mu_j}{\sigma_j} \tag{50}$$

Example: Suppose the mean and standard deviation of a feature x_j are known. You subtract the mean and divide by the standard deviation to normalize it.

- If x_1 (e.g., house size) has:

$$\mu_1 = 600, \quad \sigma_1 = 450 \tag{51}$$

  Then the standardized version is:

$$x_1^{(z)} = \frac{x_1 - 600}{450} \quad \Rightarrow \quad x_1^{(z)} \in [-0.67, 3.1] \tag{52}$$

- If x_2 (e.g., number of bedrooms) has:

$$\mu_2 = 2.3, \quad \sigma_2 = 1.4 \tag{53}$$

  Then:

$$x_2^{(z)} = \frac{x_2 - 2.3}{1.4} \quad \Rightarrow \quad x_2^{(z)} \in [-1.6, 1.9] \tag{54}$$

Practical Guidelines

- Scale all features to a similar range, ideally around $[-1, 1]$, to help gradient descent work efficiently.
- It is fine if the values are between $[-3, 3]$ or $[-0.3, 0.3]$, as long as the ranges are consistent across all features.
- If one feature ranges from -100 to 100 and another from -1 to 1, it is important to scale them to avoid imbalance.
- Very small ranges, such as $[-0.001, 0.001]$, should also be scaled to prevent numerical issues.
- Even if a feature seems reasonable, like body temperature ranging from 98.6 to 105°F, scaling it can still make gradient descent converge faster.

2.3 Checking Gradient Descent For Convergence

To fix this imbalance, input features can be rescaled so they are on similar numerical scales for example, between 0 and 1, or by standardizing them to have a mean of 0 and a standard deviation of 1.

- This makes the cost function more balanced, with contour lines that are more circular rather than stretched.

- As a result, gradient descent can move more directly toward the minimum, improving convergence speed.

Gradient descent aims to minimize the cost function $J(w, b)$. To check whether this minimization is occurring effectively, it is helpful to monitor the cost after each iteration. One common way to do this is by plotting a learning curve, which shows the value of J over successive iterations.

Learning Curve Visualization

- After each parameter update, compute the cost $J(w, b)$.
- Plot these cost values:
  - Horizontal axis: iteration number
  - Vertical axis: cost $J(w, b)$
- This plot is known as a **learning curve**.

Interpreting the Learning Curve

- A correctly functioning gradient descent algorithm produces a learning curve that consistently decreases.
- An increase in cost after an iteration typically indicates:
  - The learning rate α is too large.
  - There may be a bug in the implementation.
- Once the curve levels off, convergence is likely reached.

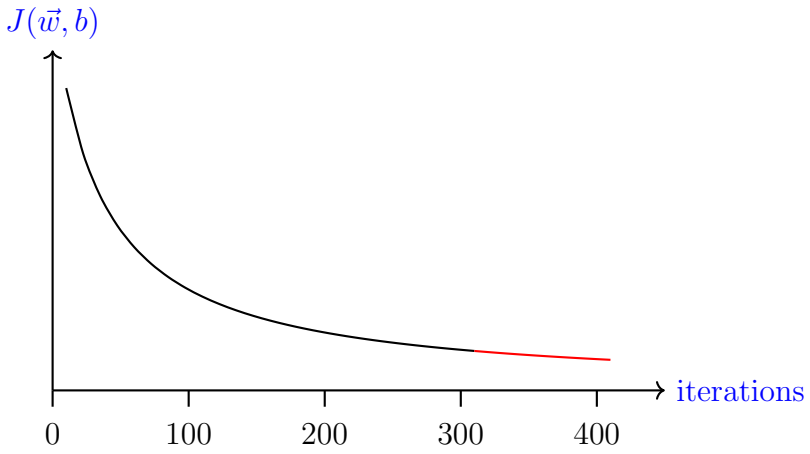


FIGURE 23. Learning curve showing cost $J(\vec{w}, b)$ over iterations of gradient descent.

Number of Iterations Required

- The number of iterations needed to converge varies widely.
- Some models converge in 30 iterations, while others may require thousands.

Optional Convergence Criterion

Define a small threshold value ε, for example $\varepsilon = 0.001$. If the reduction in cost between two consecutive iterations is less than ε, the optimization process can be stopped:

$$J^{(t)} - J^{(t+1)} < \varepsilon \tag{55}$$

This condition signals that further updates result in minimal improvement.

Note: Selecting an appropriate ε value can be difficult. Visual inspection of the learning curve is often more informative.

2.4 Choosing The Learning Rate

The learning rate α plays a critical role in how well gradient descent performs:

- If α is too small, training becomes very slow.
- If α is too large, the algorithm may overshoot the minimum or even diverge.

Detecting Problems Using the Cost Function

One way to evaluate whether gradient descent is working properly is by plotting the cost function $J(w, b)$ over iterations. Here are some behaviors to look out for:

- If the cost fluctuates up and down, gradient descent may not be converging. This could be caused by a large learning rate or a bug in the implementation.
- If the cost increases consistently, this strongly suggests that α is too large or that the update direction is wrong (e.g., using a $+\alpha$ instead of a $-\alpha$ in the update rule).

Debugging Gradient Descent

To check whether gradient descent is implemented correctly:

- Try setting α to a very small value (e.g., $\alpha = 10^{-5}$).
- If the cost still doesn't decrease consistently, there's likely a bug in the code.

This small α is only for debugging. It is not suitable for efficient training.

Finding a Suitable Learning Rate

Choosing α involves trade-offs:

- A small α ensures stability but slows training.
- A large α speeds up convergence but can cause instability.

To select a good learning rate:

(1) Try a range of values, such as:

$$\alpha \in \{0.001,\ 0.003,\ 0.01,\ 0.03,\ 0.1,\ 0.3\}$$

(2) For each value of α, run gradient descent for a small number of iterations (e.g., 100) and plot the cost curve.

(3) Choose the largest learning rate that still results in a smooth and consistent decrease in the cost.

Practical Tip

A good strategy is to:

- Start with a small value of α and increase it progressively by a factor of about 3.
- Find the largest α that results in stable and fast convergence.

Example sequence:

$$\alpha = 0.001 \rightarrow 0.003 \rightarrow 0.01 \rightarrow 0.03 \rightarrow 0.1$$

Once the value becomes too large and causes divergence or erratic cost behavior, choose a slightly smaller one.

2.5 Feature Engineering

The selection of features has a major impact on the performance of a learning algorithm. In practice, identifying or designing effective features is often essential for achieving good results.

Example: Predicting House Prices

Consider a task where the goal is to predict the price of a house using two available features:

- x_1: the width of the land plot (also known as the frontage)
- x_2: the depth of the plot

Assuming the plot is rectangular, a basic linear model could be used:

$$f_{\vec{w},b}(x) = w_1 x_1 + w_2 x_2 + b \tag{56}$$

This model treats width and depth as independent inputs. However, it may be more informative to consider the total area of the plot. The area of a rectangular plot can be calculated as:

$$x_3 = x_1 \cdot x_2 \tag{57}$$

This new feature x_3 represents the land area and may be a better indicator of price than the width or depth alone. With this, the model can be extended as follows:

$$f_{\vec{w},b}(x) = w_1x_1 + w_2x_2 + w_3x_3 + b \tag{58}$$

Now the model can decide, based on the training data, whether frontage, depth, or area is more useful for predicting the price.

This process of constructing new features from existing ones is called **feature engineering**. It involves using domain knowledge or logical combinations of features to help the model discover better patterns. Typical goals of feature engineering include:

- Providing more informative inputs to the model
- Allowing the algorithm to learn complex relationships more easily
- Improving accuracy and generalization

Instead of relying only on the original inputs, thoughtfully engineered features often lead to simpler and more effective models.

2.6 Polynomial Regression

Many real-world problems require fitting curves. Polynomial regression builds on linear regression by adding new features that allow the model to fit non-linear patterns.

Why Polynomial Features?

Suppose the input feature x is the size of a house (in square feet), and the target is the house price. A straight line may not fit the data well.

$$f(x) = w_1x + b \tag{59}$$

This model might not capture the curved nature of the data. A better option could be a quadratic function:

$$f(x) = w_1x + w_2x^2 + b \tag{60}$$

However, a quadratic curve eventually turns downward. This may not make sense for prices, which should keep increasing with size. A cubic function can fix that:

$$f(x) = w_1x + w_2x^2 + w_3x^3 + b \tag{61}$$

This is an example of **polynomial regression**, where the model includes powers of the input feature.

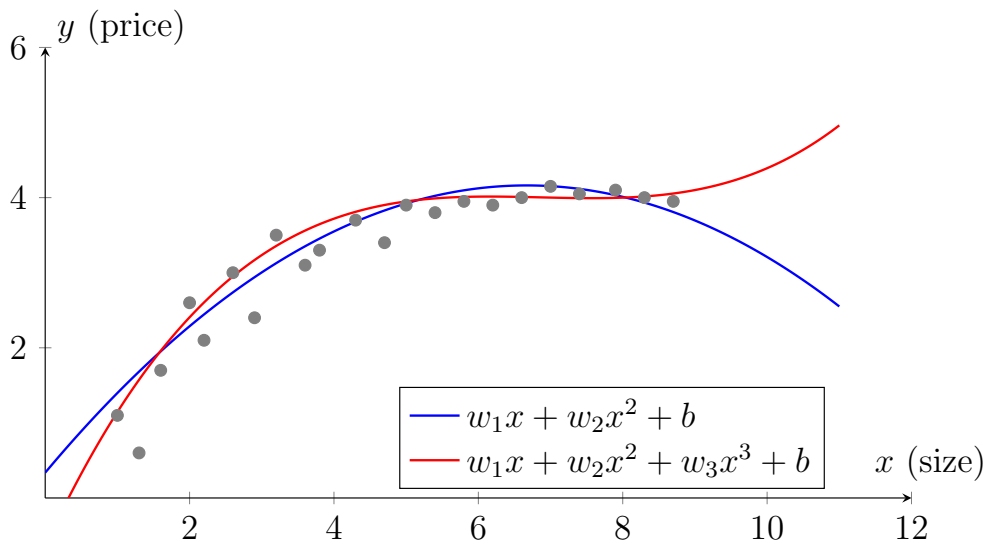


FIGURE 24. Quadratic vs. cubic polynomial fits to housing price–size data.

Creating Polynomial Features

To use polynomial regression, new features are built from the original input x. These include powers of x, such as:

$$x_1 = x, \quad x_2 = x^2, \quad x_3 = x^3 \tag{62}$$

Using these features, the model becomes:

$$f(x) = w_1x_1 + w_2x_2 + w_3x_3 + b \tag{63}$$

This lets the model fit curves instead of just straight lines, helping it learn more complex (non-linear) patterns in the data.

Feature Scaling Is Essential

When using higher-order features, their values can be much larger than the original input. For example:

- If $x \in [1, 1000]$, then:
  - $x^2 \in [1, 10^6]$
  - $x^3 \in [1, 10^9]$

These large differences can make gradient descent unstable. Feature scaling helps normalize values so they are on similar scales.

Other Transformations

Instead of polynomial terms, other transformations can also be used. For example, the square root of x:

$$f(x) = w_1 x + w_2 \sqrt{x} + b \tag{64}$$

The square root function grows slowly and may be a better fit in some situations.

Choosing Features

Selecting the right features depends on understanding the problem and experimenting with different options.

- Try combinations like x, x^2, x^3, or $\sqrt{x}$
- Compare how well different models fit the data
- Pick the set of features that improves accuracy without making the model too complex

Module 3: Classification

1 Classification With Logistic Regression

1.1 Motivations

In regression tasks, the target variable y is a continuous number. For example, predicting the price of a house or the temperature outside.

But in many situations, the goal is to predict categories instead of numbers. For example, determining whether an email is spam or whether a tumor is cancerous. These problems are called **classification** tasks.

Binary Classification

In **binary classification**, the label y can take on only two values:

- Email classification: spam (1) or not spam (0)
- Fraud detection: fraudulent (1) or legitimate (0)
- Tumor classification: malignant (1) or benign (0)

By convention:

- $y = 1$: *positive class* (e.g., spam, fraud, malignant)
- $y = 0$: *negative class* (e.g., not spam, normal, benign)

These labels just represent whether a condition is present (1) or not (0). They do not mean good or bad.

Why Not Use Linear Regression for Classification?

Suppose the input is tumor size x, and the goal is to predict whether the tumor is malignant. If linear regression is used, it gives a prediction based on the equation:

$$f(x) = wx + b \tag{65}$$

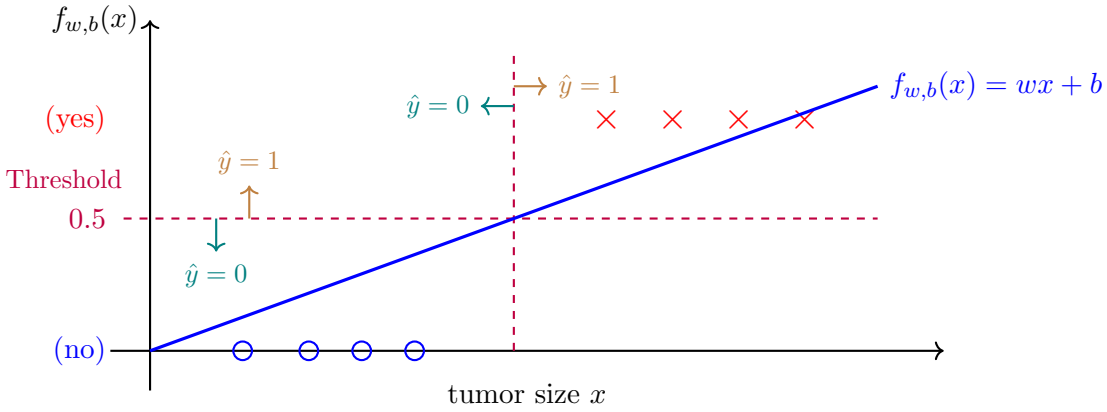
To make binary predictions (e.g., predicting whether a tumor is malignant), a threshold can be used to convert the continuous output into a classification:

$$\hat{y} = \begin{cases} 1 & \text{if } f(x) \geq 0.5 \\ 0 & \text{if } f(x) < 0.5 \end{cases} \tag{66}$$

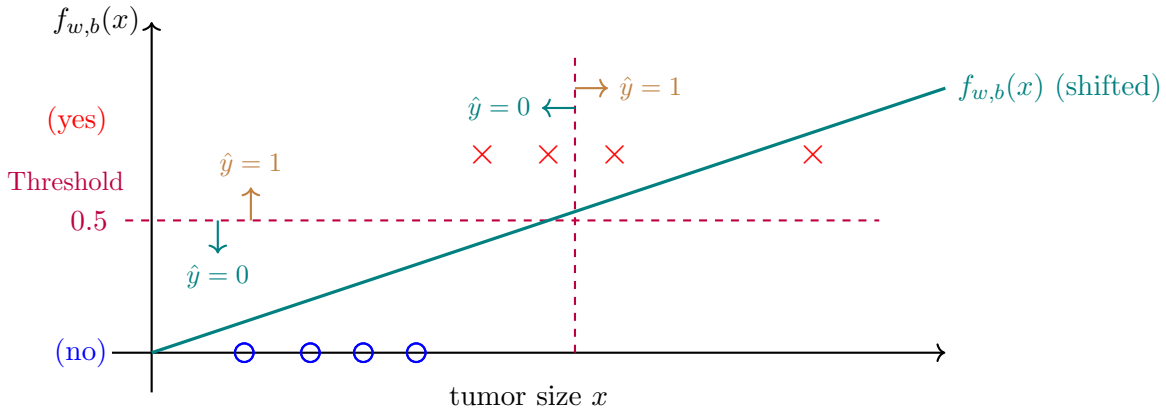
In this case, anything above 0.5 is classified as malignant (1), and anything below is benign (0).

Figure 25(A) shows this idea. The diagonal line represents the linear regression model. A dashed horizontal line shows the threshold at 0.5, and a vertical line shows the point where the decision switches from class 0 to class 1.

However, this approach has a problem: if a single new point is added far to the right, the line can shift, causing the threshold to change. This may lead to incorrect predictions, as shown in Figure 25(B). Therefore, linear regression is not reliable for classification.



(A) Linear regression with a threshold (0.5) for binary classification.



(B) A distant outlier shifts the regression line and threshold, leading to incorrect predictions.

FIGURE 25. Linear regression applied to binary classification. While a threshold can be used, the model is sensitive to outliers, which can distort the decision boundary.

1.2 Logistic Regression

Logistic regression is one of the most commonly used classification algorithms. It is especially useful for **binary classification**, where the output label y can only take two possible values. For example:

- $y = 1$ (e.g., malignant tumor)
- $y = 0$ (e.g., benign tumor)

Suppose the input feature is tumor size x. In linear regression, a straight line is fitted, which can give outputs outside the range $[0, 1]$. This is not ideal for classification. Logistic regression solves this by using a special function called the **sigmoid function**, which maps any input to a value between 0 and 1.

The Sigmoid (Logistic) Function

The sigmoid function is defined as:

$$g(z) = \frac{1}{1 + e^{-z}} \tag{67}$$

This function has the following properties:

- If z is a large positive number (e.g., $z = 100$), then $g(z) \approx 1$
- If z is a large negative number (e.g., $z = -100$), then $g(z) \approx 0$
- If $z = 0$, then $g(0) = 0.5$

This makes the sigmoid function ideal for representing probabilities, since its outputs are always in the range $[0, 1]$.

Figure 26 shows how logistic regression uses a sigmoid (S-shaped) curve to model the probability that a tumor is malignant based on its size. The curve smoothly increases from 0 to 1, mapping small tumors to low malignancy probability and larger tumors to higher probability. Data points at $y = 0$ represent benign cases, and those at $y = 1$ represent malignant ones.

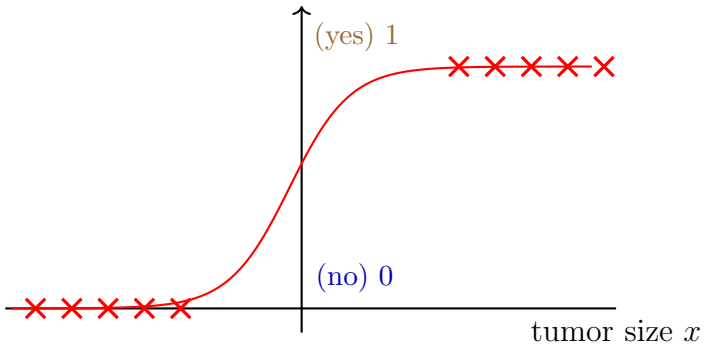


FIGURE 26. Sigmoid curve estimating malignancy probability from tumor size.

Logistic Regression Model

Logistic regression combines a linear model with the sigmoid function. The steps are:

(1) Compute a linear combination of the input:

$$z = w \cdot x + b \tag{68}$$

(2) Apply the sigmoid function to get the prediction:

$$f(x) = g(z) = \frac{1}{1 + e^{-(w \cdot x + b)}} \tag{69}$$

The output $f(x)$ represents the estimated probability that the label y is 1 (e.g., malignant), given the input x:

$$f(x) = P(y = 1 \mid x; w, b) \tag{70}$$

Note: The semicolon in $P(y = 1 \mid x; w, b)$ indicates that the probability depends on the input x, given the model parameters w and b.

Making Predictions

Since the model outputs a probability between 0 and 1, a threshold is used to make a binary decision. A common threshold is 0.5:

$$\hat{y} = \begin{cases} 1 & \text{if } f(x) \geq 0.5 \\ 0 & \text{if } f(x) < 0.5 \end{cases} \tag{71}$$

Example: If the model outputs $f(x) = 0.7$ for a tumor of a certain size, this means:

- 70% probability that the tumor is malignant ($y = 1$)
- 30% probability that it is benign ($y = 0$)

So, the prediction would be malignant.

1.3 Decision Boundary

Logistic regression predicts outcomes in two steps:

(1) Compute a linear combination of the inputs:

$$z = w \cdot x + b \tag{72}$$

(2) Apply the sigmoid function to obtain a probability:

$$f(x) = g(z) = \frac{1}{1 + e^{-z}} \tag{73}$$

The output $f(x) \in [0, 1]$ is interpreted as the probability that $y = 1$ given x, with model parameters w and b.

Making a Prediction

To classify an example, a threshold (commonly 0.5) is applied:

$$\hat{y} = \begin{cases} 1 & \text{if } f(x) \geq 0.5 \\ 0 & \text{if } f(x) < 0.5 \end{cases} \tag{74}$$

Because $f(x) = g(z)$, and $g(z) \geq 0.5$ when $z \geq 0$, this threshold rule is equivalent to:

$$\hat{y} = \begin{cases} 1 & \text{if } w \cdot x + b \geq 0 \\ 0 & \text{if } w \cdot x + b < 0 \end{cases} \tag{75}$$

This inequality defines the **decision boundary**, which is the surface that separates inputs classified as class 0 from those classified as class 1.

Example: Two Features (Linear Boundary)

If the input has two features, x_1 and x_2, the model computes:

$$z = w_1 x_1 + w_2 x_2 + b \tag{76}$$

With parameters $w_1 = 1$, $w_2 = 1$, and $b = -3$, the decision boundary occurs when:

$$x_1 + x_2 = 3 \tag{77}$$

This is a straight line separating the two classes.

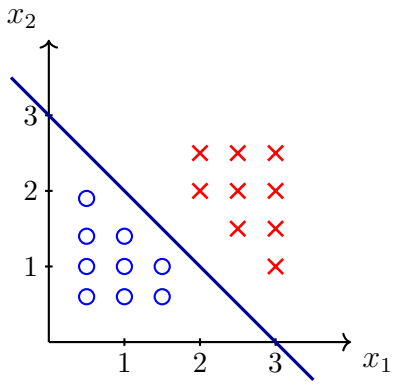


FIGURE 27. Linear decision boundary for logistic regression: $x_1 + x_2 = 3$.

Nonlinear Boundary with Polynomial Features

More complex decision boundaries can be learned by adding polynomial terms. For instance:

$$z = w_1 x_1^2 + w_2 x_2^2 + b \tag{78}$$

If $w_1 = w_2 = 1$, $b = -1$, then the decision boundary becomes:

$$x_1^2 + x_2^2 = 1 \tag{79}$$

This describes a circular boundary.

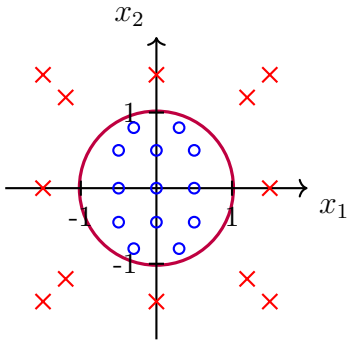


FIGURE 28. Nonlinear decision boundary: $x_1^2 + x_2^2 = 1$.

More Flexible Boundaries with Higher-Degree Polynomials

To allow for even more complex boundaries, higher-degree polynomial terms can be added:

$$z = w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_1 x_2 + w_5 x_2^2 \tag{80}$$

The shape of the decision boundary depends on the learned parameters. For example, this may result in elliptical or more intricate curved regions separating class 0 from class 1.

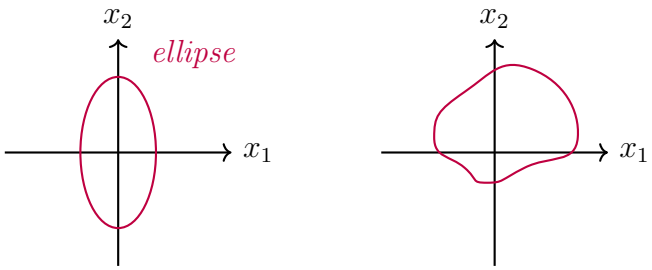


FIGURE 29. Decision boundary shaped by higher-degree polynomial terms.

2 Cost Function For Logistic Regression

2.1 Cost Function For Logistic Regression

The cost function measures how well the model parameters w and b fit the training data. A suitable cost function helps find parameters that reduce the prediction error.

Why Not Use Squared Error?

In linear regression, the mean squared error (MSE) cost function is used:

$$J(\vec{w}, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f(\vec{x}^{(i)}) - y^{(i)} \right)^2 \tag{81}$$

This function is convex, meaning it has a single global minimum. Gradient descent can reliably optimize this function. In contrast, logistic regression models the output as a probability:

$$f(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}} \tag{82}$$

If the squared error cost function is used here, the resulting cost surface becomes non-convex with many local minima. This makes optimization with gradient-based methods unreliable.

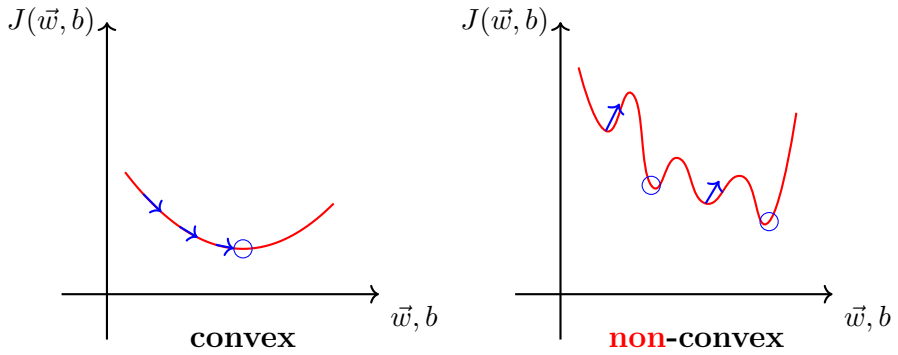


FIGURE 30. Comparison of convex and non-convex cost functions. Convex functions have a single global minimum, making them easier to optimize. Non-convex functions may have multiple local minima, making optimization more challenging.

A Better Loss Function for Logistic Regression

To ensure convexity, logistic regression uses a different loss function derived from the likelihood of the Bernoulli distribution. For a single training example, the loss function is:

$$\mathcal{L}(f(\vec{x}), y) = \begin{cases} -\log(f(\vec{x})) & \text{if } y = 1 \\ -\log(1 - f(\vec{x})) & \text{if } y = 0 \end{cases} \tag{83}$$

This loss behaves differently depending on the true label y and the predicted probability $f(\vec{x})$.

- **When $y = 1$:** The true label is positive. The model is expected to predict a value close to 1.
  - If $f(\vec{x})$ is close to 1 (correct and confident), then:
$$\mathcal{L}(f(\vec{x}), 1) = -\log(f(\vec{x})) \approx 0$$
  - If $f(\vec{x})$ is close to 0 (confident but wrong), then:
$$\mathcal{L}(f(\vec{x}), 1) = -\log(f(\vec{x})) \to \infty$$
- **When $y = 0$:** The true label is negative. The model is expected to predict a value close to 0.
  - If $f(\vec{x})$ is close to 0 (correct and confident), then:
$$\mathcal{L}(f(\vec{x}), 0) = -\log(1 - f(\vec{x})) \approx 0$$
  - If $f(\vec{x})$ is close to 1 (confident but wrong), then:
$$\mathcal{L}(f(\vec{x}), 0) = -\log(1 - f(\vec{x})) \to \infty$$

Summary:

- Correct predictions lead to small loss values.
- Wrong predictions lead to large loss values, especially when the model is confident.
- The function encourages the model to output probabilities close to the correct class.

Cost Function Over the Dataset

The total cost over all m training examples is computed by averaging the individual losses:

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(f(\vec{x}^{(i)}), y^{(i)}) \tag{84}$$

This cost function is convex, making it well-suited for optimization via gradient descent.

2.2 Simplified Cost Function for Logistic Regression

In logistic regression, the task is binary classification, where the target label $y \in \{0, 1\}$. The original loss function is defined piecewise:

$$\mathcal{L}(f(\vec{x}), y) = \begin{cases} -\log(f(\vec{x})) & \text{if } y = 1 \\ -\log(1 - f(\vec{x})) & \text{if } y = 0 \end{cases} \tag{85}$$

Simplified Loss Function (Single Formula)

Because y is either 0 or 1, the loss can be written as a single equation:

$$\mathcal{L}(f(\vec{x}), y) = -\left[y \cdot \log(f(\vec{x})) + (1-y) \cdot \log(1-f(\vec{x}))\right] \tag{86}$$

Why this works:

- If $y = 1$, then $(1-y) = 0$, and the loss becomes:

$$\mathcal{L} = -\log(f(\vec{x})) \tag{87}$$

- If $y = 0$, then $y = 0$, and the loss becomes:

$$\mathcal{L} = -\log(1-f(\vec{x})) \tag{88}$$

- This confirms that the simplified formula correctly represents both cases.

Cost Function for the Dataset

The cost function $J(\vec{w}, b)$ is the average of the loss across all m training examples:

$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(f(\vec{x}^{(i)}), y^{(i)}) \tag{89}$$

Substituting the simplified loss expression:

$$\boxed{J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \cdot \log(f(\vec{x}^{(i)})) + (1-y^{(i)}) \cdot \log(1-f(\vec{x}^{(i)}))\right]} \tag{90}$$

This is the standard cost function used in logistic regression training.

Justification for This Cost Function

- This function is derived using the statistical principle of **maximum likelihood estimation**.
- It is **convex**, which ensures that gradient descent will converge to a global minimum.
- It is efficient to implement and differentiable.

3 Gradient Descent For Logistic Regression

3.1 Gradient Descent Implementation

The goal in logistic regression is to find parameters $\vec{w}$ and b that minimize the cost function $J(\vec{w}, b)$. Once trained, the model can be used to predict the probability that a new input $\vec{x}$ belongs to class $y = 1$. For example, in a medical application, the model could estimate the probability that a patient has a disease based on features such as tumor size and age.

Optimization Using Gradient Descent

To minimize the cost function, gradient descent is applied. The update rule for each parameter is:

$$w_j := w_j - \alpha \cdot \frac{\partial J(\vec{w}, b)}{\partial w_j} \quad \text{and} \quad b := b - \alpha \cdot \frac{\partial J(\vec{w}, b)}{\partial b} \tag{91}$$

Let $f(\vec{x}^{(i)})$ denote the prediction on the i-th training example. Then:

$$\frac{\partial J(\vec{w}, b)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left(f(\vec{x}^{(i)}) - y^{(i)} \right) x_j^{(i)} \quad \text{and} \quad \frac{\partial J(\vec{w}, b)}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \left(f(\vec{x}^{(i)}) - y^{(i)} \right) \tag{92}$$

These expressions are similar to those used in linear regression, but the function $f(\vec{x})$ differs.

Difference from Linear Regression

Although the gradient formulas appear similar to those in linear regression, logistic regression uses a different hypothesis function:

$$f(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}} \tag{93}$$

This is the sigmoid function, which outputs probabilities between 0 and 1. In contrast, linear regression uses:

$$f(\vec{x}) = \vec{w} \cdot \vec{x} + b \tag{94}$$

Thus, despite structural similarities in gradient descent updates, the models behave differently due to the use of different functions $f(\vec{x})$.

Implementation Tips

- Gradient updates should be performed simultaneously for all parameters.
- Vectorized implementations can significantly speed up training, as shown in the optional lab materials.
- Feature scaling (e.g., normalizing features to range $[-1, 1]$) helps improve convergence speed.

4 The Problem Of Overfitting

4.1 The Problem Of Overfitting

Linear and logistic regression are effective algorithms for many tasks. However, they can suffer from two common problems: **underfitting** and **overfitting**. These issues affect model performance and generalization.

Underfitting (High Bias) A model is said to underfit when it is too simple to capture the underlying patterns in the data. For example, fitting a linear model to a dataset where the true relationship is nonlinear results in poor performance on both the training and test sets.

- The model cannot represent the complexity of the data.
- It performs poorly even on the training data.
- Technically referred to as **high bias**.

Example: In a housing price prediction task, a linear model may fail to capture the flattening of prices at larger house sizes, leading to underfitting.

Overfitting (High Variance) Overfitting occurs when a model is too complex and fits the training data too closely, including noise. While it performs well on the training set, it generalizes poorly to new, unseen data.

- The model captures noise in the training data.
- It shows low training error but high test error.
- Technically referred to as **high variance**.

Example: Fitting a fourth-order polynomial to a small training set might result in a curve that passes through all training points, but fails to make reliable predictions on new examples.

Good Fit (A Balanced Model) A well-performing model balances complexity and generalization. It should fit the training data reasonably well without overfitting.

Example: A quadratic model that captures the data trend without fitting every point perfectly often generalizes better than a higher-order polynomial.

FIGURE 31. Examples of underfitting, good fit, and overfitting. The good fit strikes a balance between bias and variance, while the others either oversimplify or overreact to training data.

Bias and Variance In machine learning, the terms **bias** and **variance** are used to describe underfitting and overfitting respectively.

Summary

- **Underfitting** (high bias): Too simple, poor training and test performance.
- **Overfitting** (high variance): Too complex, low training error, high test error.
- **Goal:** Find a model with a good balance (one that generalizes well).

4.2 Addressing Overfitting

When a model exhibits high variance and overfits the training data, several strategies can be used to mitigate the problem:

- **Collect More Training Data:**
  - More data helps the model learn a less complex function that generalizes better.
  - A large training set can reduce the tendency of high-degree polynomials to overfit.
  - This is often the most effective solution, but not always feasible.

- **Use Fewer Features (Feature Selection):**
  - Reducing the number of input features can simplify the model.
  - Instead of using $x, x^2, x^3, \ldots, x^n$, select only the most relevant ones.
  - Alternatively, if you have many real-world features (e.g., size, bedrooms, age), selecting a subset (e.g., size, bedrooms) may reduce overfitting.
  - This process is known as *feature selection*.
  - Be cautious: removing features means potentially discarding useful information.

- **Apply Regularization:**
  - Regularization discourages the model from learning large parameter values.
  - Instead of setting feature weights to exactly zero, regularization penalizes large values of w_j.
  - This allows all features to remain in the model but limits their impact.
  - Usually only the weights $w_1, \ldots, w_n$ are regularized; the bias term b is often left unregularized.
  - Regularization is commonly used in practice, especially in complex models like neural networks.

4.3 Cost Function With Regularization

Regularization is a technique used to reduce overfitting by discouraging overly complex models. This is achieved by modifying the cost function to penalize large weights.

- A high-degree polynomial may fit the training data perfectly but result in poor generalization.
  - ***Example:*** Adding x^3 and x^4 terms can cause a *wiggly* fit, overfitting the data.

- ***Solution:*** Penalize large parameters such as w_3 and w_4 to make the model simpler.

Modified Cost Function

Standard cost for linear regression:

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 \tag{95}$$

Regularized cost function:

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2 \tag{96}$$

- λ is the **regularization parameter**, which controls the strength of the penalty.
- The term $\frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2$ is called the **regularization term**.
- Regularization discourages large weights, making the model smoother and reducing overfitting.
- Conventionally, b (the bias term) is not regularized.

Effect of Different λ Values

- $\lambda = 0$: No regularization. Risk of overfitting due to complex, high-variance models.
- $\lambda \to \infty$: Heavy regularization forces all $w_j \to 0$. Model underfits and becomes overly simple.
- **Ideal range:** A moderate λ achieves a good trade-off between bias and variance.

4.4 Regularized Linear Regression

Regularization addresses overfitting by penalizing large parameter values.

Regularized Cost Function

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 + \frac{\lambda}{2m}\sum_{j=1}^{n} w_j^2 \tag{97}$$

Where

- m is the number of training examples
- λ is the regularization parameter
- $\hat{y}^{(i)} = w^T x^{(i)} + b$ is the model prediction

- The second term penalizes large weight values (w_j)

Note: The bias term b is not regularized.

Gradient Descent Updates

To minimize the cost function, gradient descent updates the weights and bias as follows:

$$w_j := w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right] \quad \text{and} \quad b := b - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) \right] \tag{98}$$

Remarks:

- The bias term is not affected by regularization.
- All parameters must be updated simultaneously.

4.5 Regularized Logistic Regression

Logistic regression models can overfit the training data, especially when using a large number of features. For example, high-degree polynomial terms can result in overly complex decision boundaries that perform well on the training set but poorly on unseen data.

To address this, regularization is introduced by modifying the cost function to penalize large parameter values.

Regularized Cost Function

The standard logistic regression cost function is:

$$J(w, b) = \frac{1}{m} \sum_{i=1}^{m} \left[-y^{(i)} \log(\hat{y}^{(i)}) - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] \tag{99}$$

To regularize the model, an additional penalty term is added:

$$J(w, b) = \frac{1}{m} \sum_{i=1}^{m} \left[-y^{(i)} \log(\hat{y}^{(i)}) - (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2 \tag{100}$$

Where

- $\hat{y}^{(i)} = \sigma(w^T x^{(i)} + b)$, with $\sigma(z)$ being the sigmoid function
- λ is the regularization parameter
- w_j represents the weights (excluding b)

Note: The bias term b is not regularized.

Gradient Descent Updates

To minimize the regularized cost function, gradient descent is used. The updates are as follows:

For each weight w_j (where $j = 1, 2, \ldots, n$):

$$w_j := w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right] \tag{101}$$

For the bias term b:

$$b := b - \alpha \left[\frac{1}{m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)}) \right] \tag{102}$$

Effect of Regularization

- The regularization term $\frac{\lambda}{m} w_j$ shrinks the weights slightly during each iteration, helping to reduce model complexity.
- This discourages the learning algorithm from fitting overly complex models, thus improving generalization on unseen data.
- Regularization does not affect the bias term b.